

BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: ANDREW LEYIAPAS

Admission Number: BIT/2024/74067

School/Faculty: COMPUTER AND INFORMATICS

Programme: MOBILE APPLICATION

DEVELOPMENT

Academic Year: 2026

Semester: AUG/SEP

Lecturer: MICHAEL NYORO

Phone Number: 0111579951

Instructions

This logbook is used to document weekly learning activities, practical exercises, skills acquired, challenges encountered, solutions applied, and reflections. Students should complete all sections every week.

Each week to follow remarks in week 1 section.

Autogenerated Table of content

Week 1: Introduction to Mobile Application Development

Practical Activities Undertaken

- Installed Android Studio and configured development environment
- Created first Android project (STUDENT PORTALPRO)
- Learned basic structure of Android applications (Activities, XML layouts)
- Ran application on emulator for testing setup

Screenshots/Evidence Attached

only screenshots needed and labeling

Screenshots/Evidence Attached (Yes/No)

Android Studio installation/setup

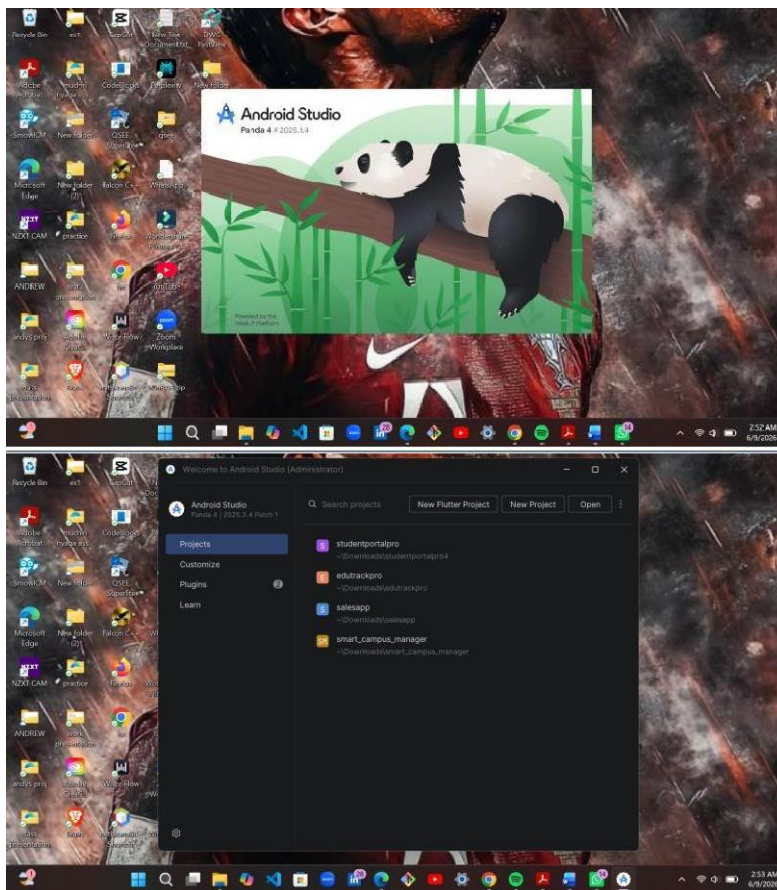


Figure 1. ANDROID STUDIO.

INSTALLATIO

Created project structure

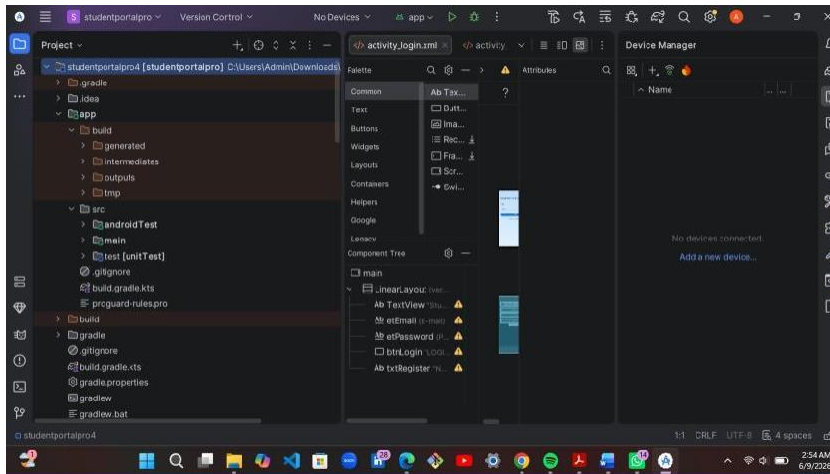


Figure 2.STUDENT PORTAL

PRO PROJECT CREATION

Emulator running

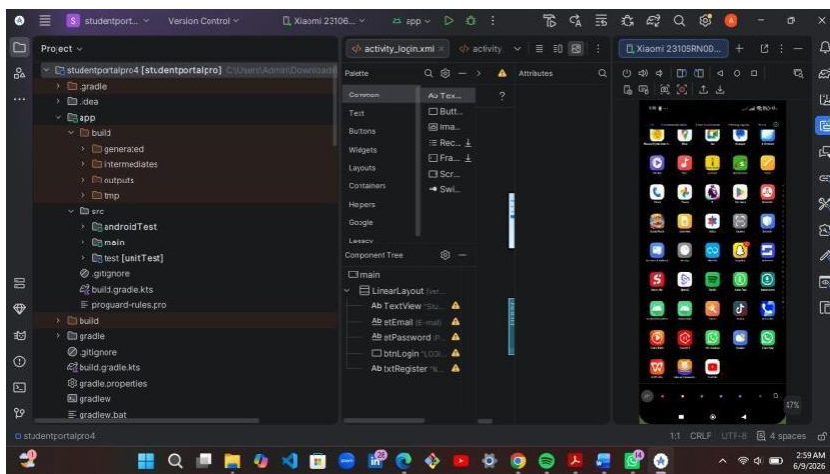


Figure 3.EMULATOR APP

RUNNING

Tools/Software Used

1. Android Studio
2. Java
3. Android Emulator
4. Windows OS

Personal Reflection

This week I learned how to set up Android Studio and create my first mobile application. I understood the basic project structure and how the emulator works for testing apps.

Week 2: Mobile Development Environment

Practical Activities Undertaken

- Designed login screen using XML
- Added Edit Text fields for username and password
- Created buttons for login and register
- Applied basic Constraint Layout for alignment

Screenshots/Evidence Attached

Login screen design

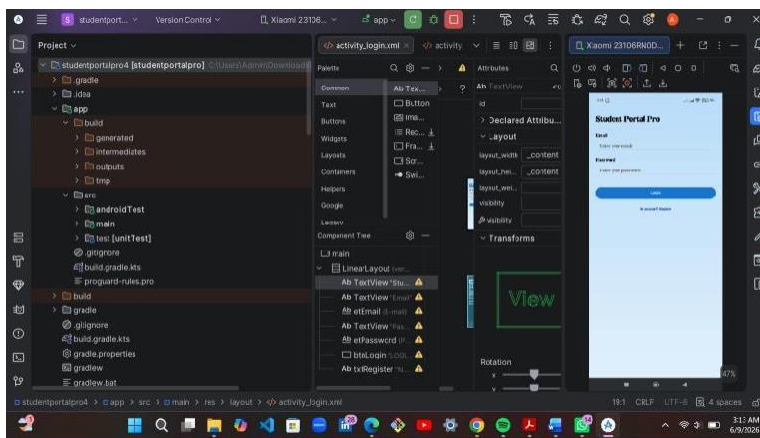
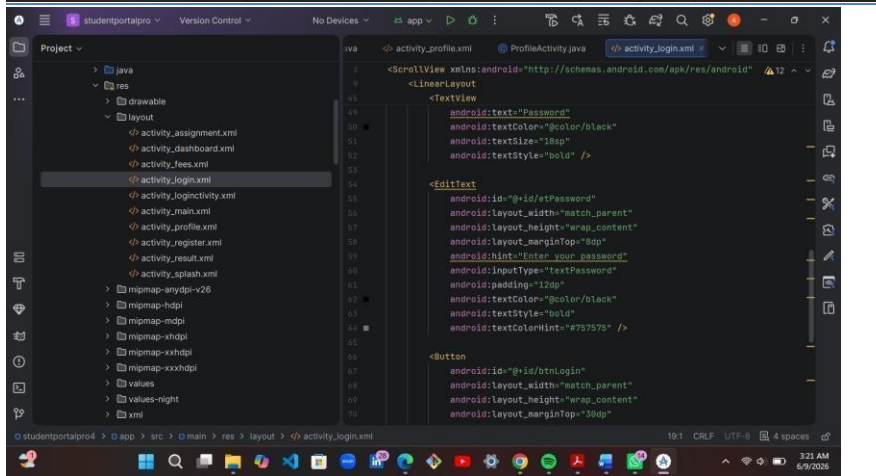
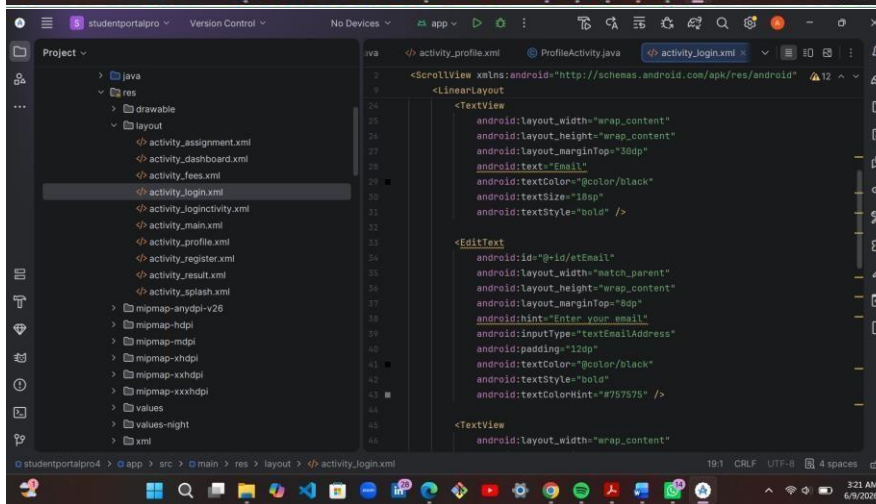
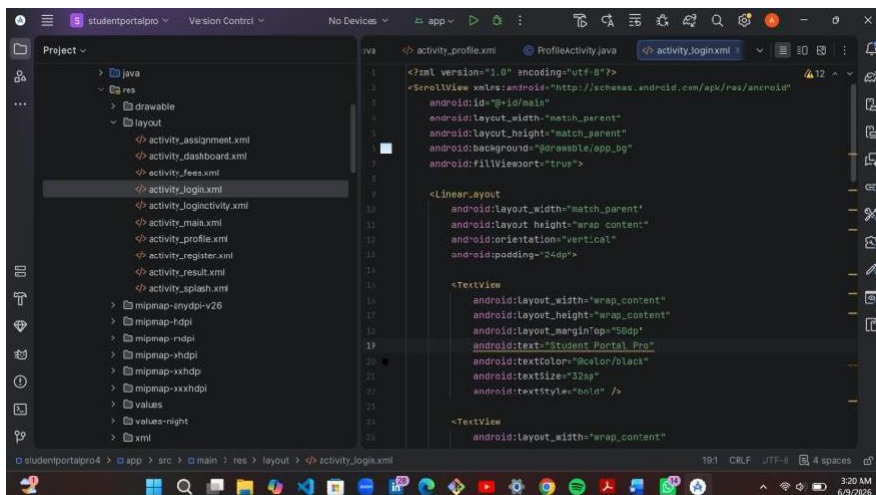


Figure 4.LOGIN SCREENS DESIGN

XML layout code



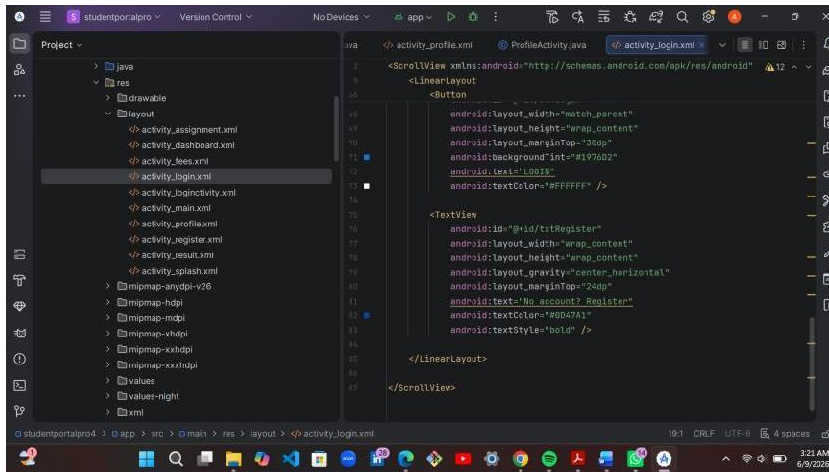


Figure 5.WHOLE

ACTIVITY LOGIN.XML CODE

Running UI on emulator

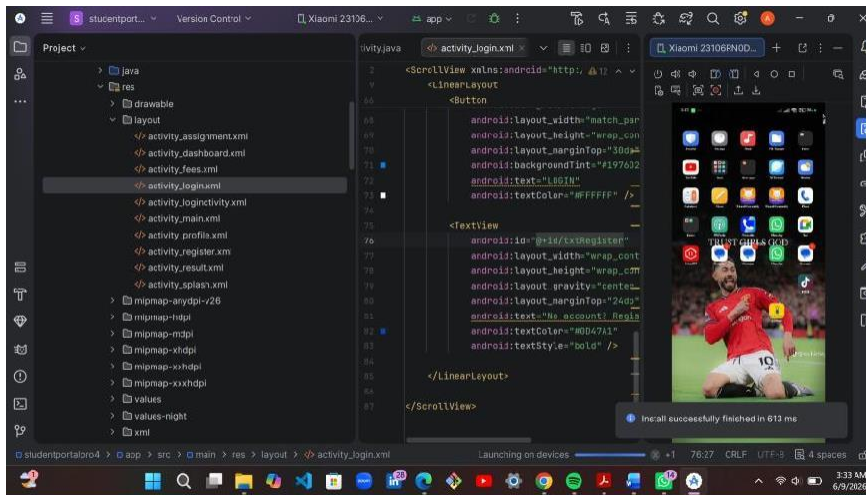


Figure 6.APP LAUNCHING

ON DEVICE

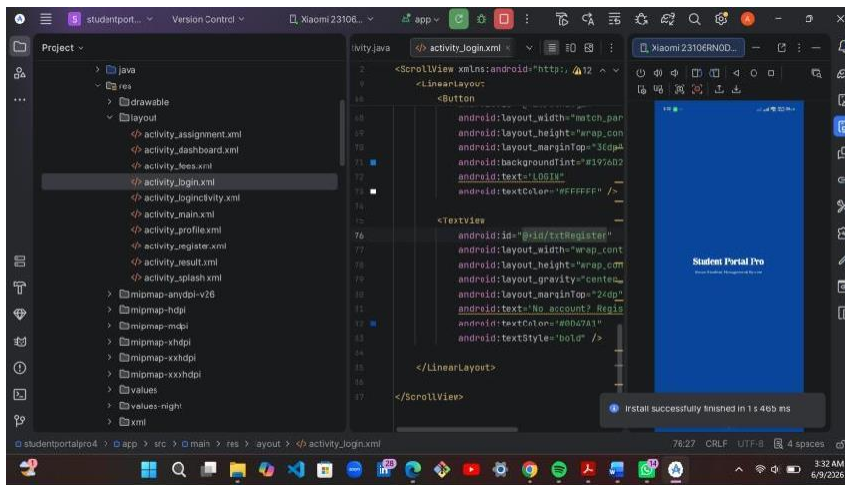


Figure 7.SUCCESSFULL

INSTALLATION

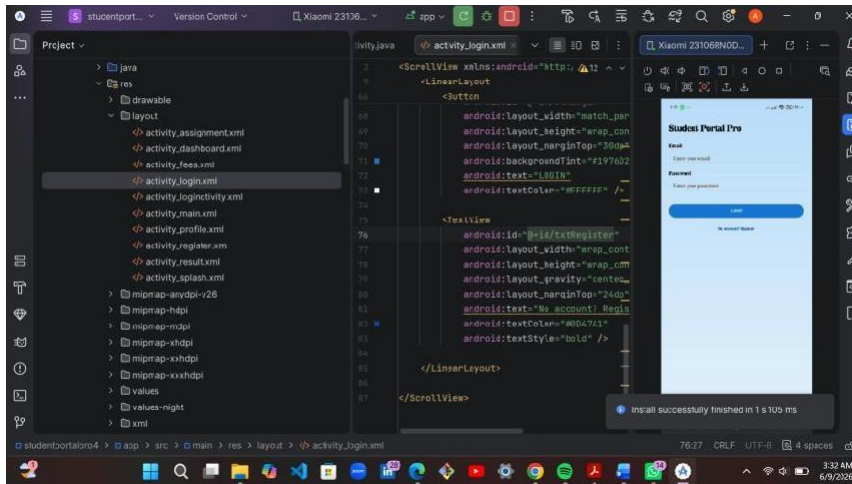


Figure 8.LOGIN SCREEN

ON EMULATOR VIEW

Tools/Software Used

- Android Studio
- XML
- Constraint Layout

Personal Reflection

This week I learned how to design user interfaces using XML in Android Studio. I also improved my understanding of layouts and positioning of UI components.

Week 3: User Interface Design

Practical Activities Undertaken

- Created dashboard screen for users
- Developed navigation between activities
- Added buttons for “Report Lost Item” and “Report Found Item”
- Improved UI design for better user experience

Dashboard screen

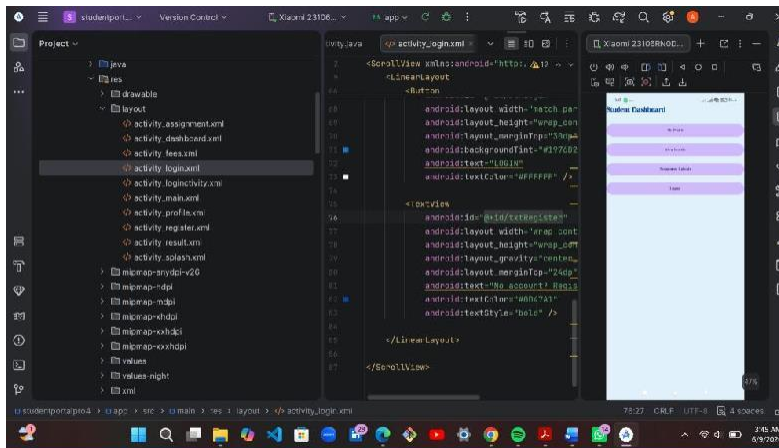


Figure 9. DASHBOARD SCREENS

Navigation between pages

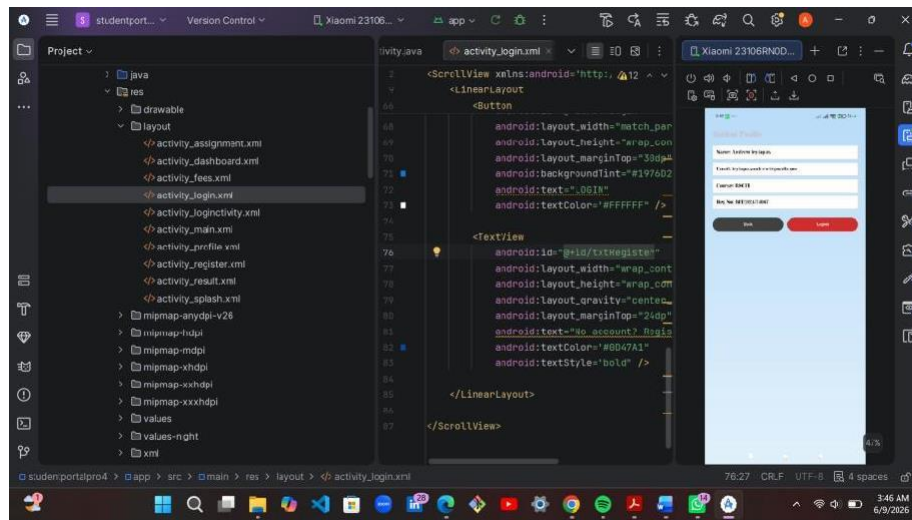


Figure 10.PROFILE BUTTON

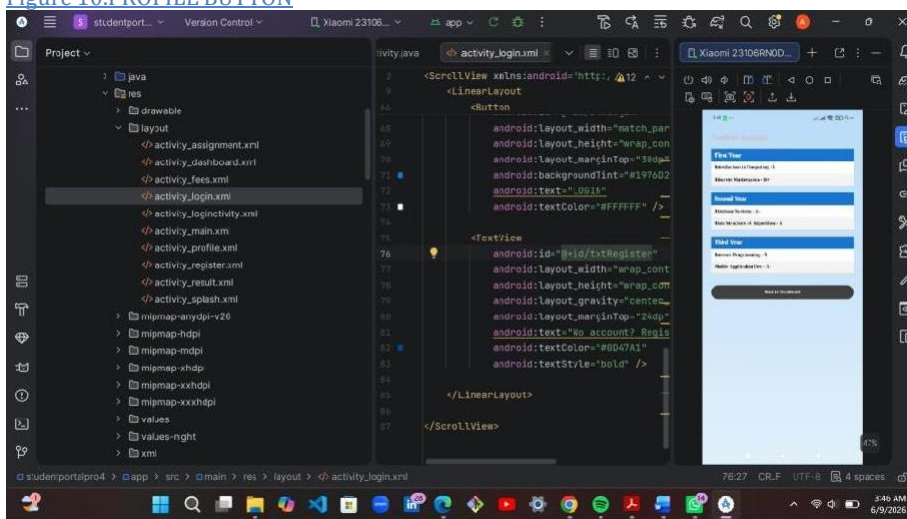


Figure 11.RESULTS BUTTON

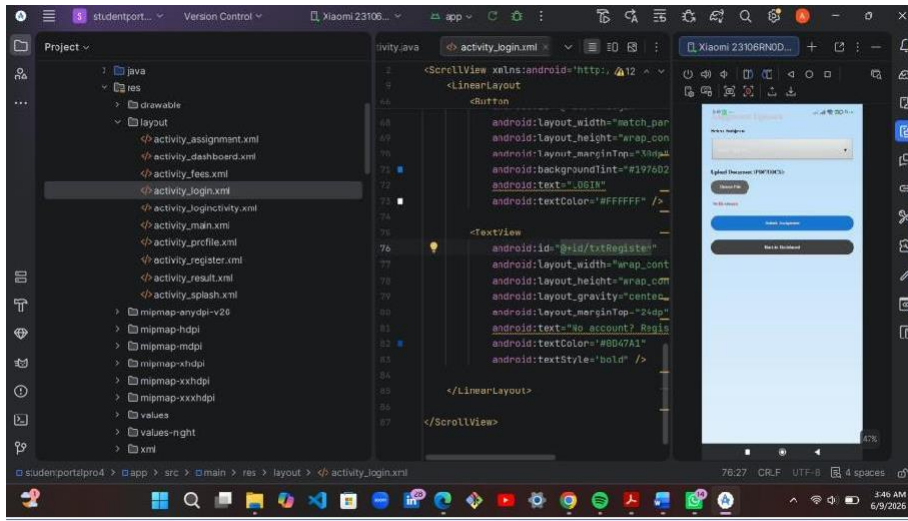
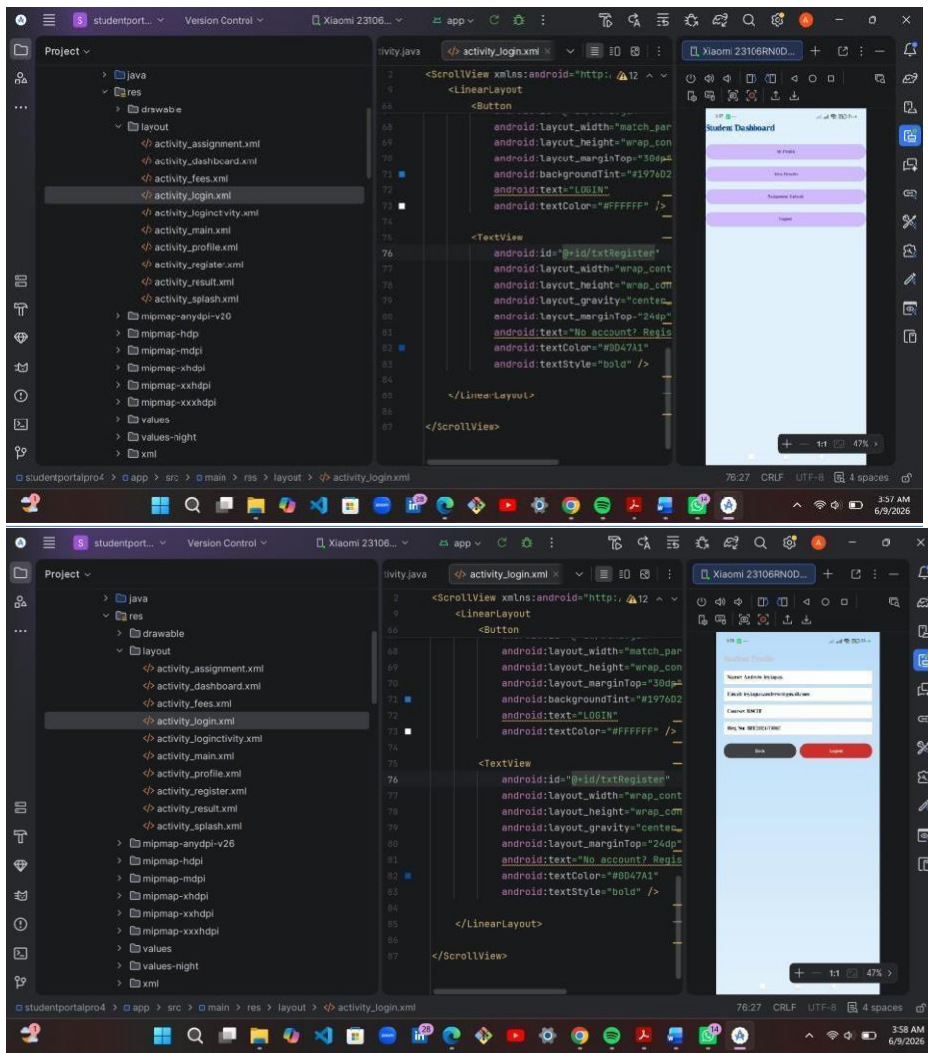


Figure 12.ASSIGNMENT BUTTON

Button interaction



Figure

13.DASHBOARD SCREENS&PROFILE BUTTONS&PROFILE SCREENS

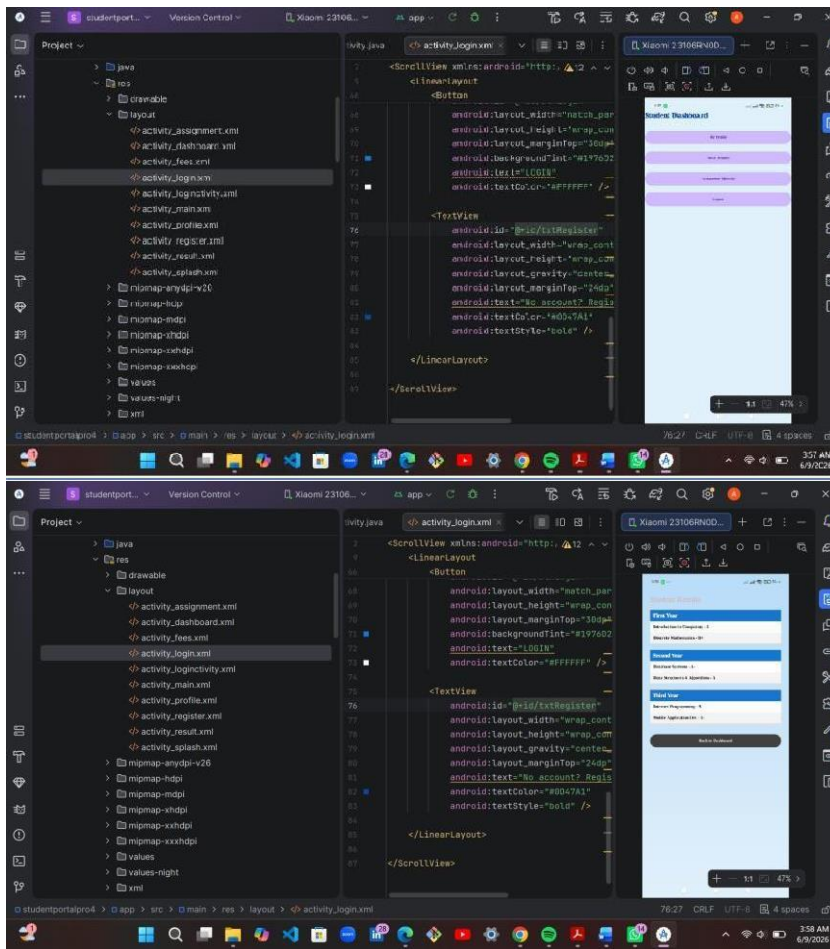
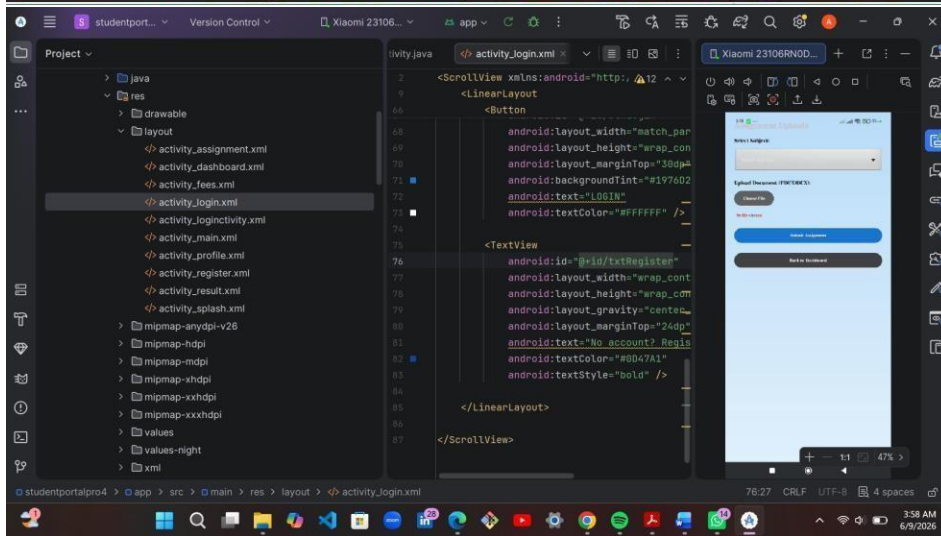
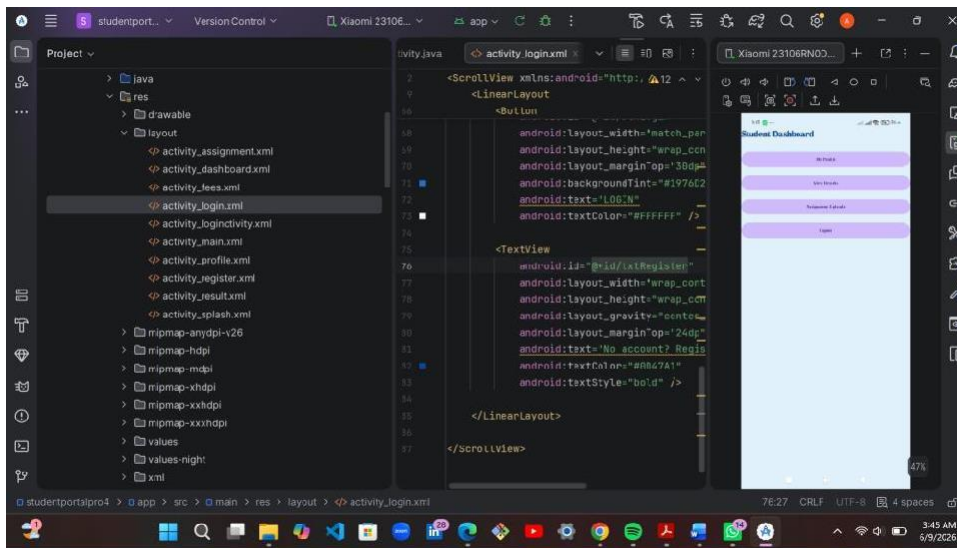


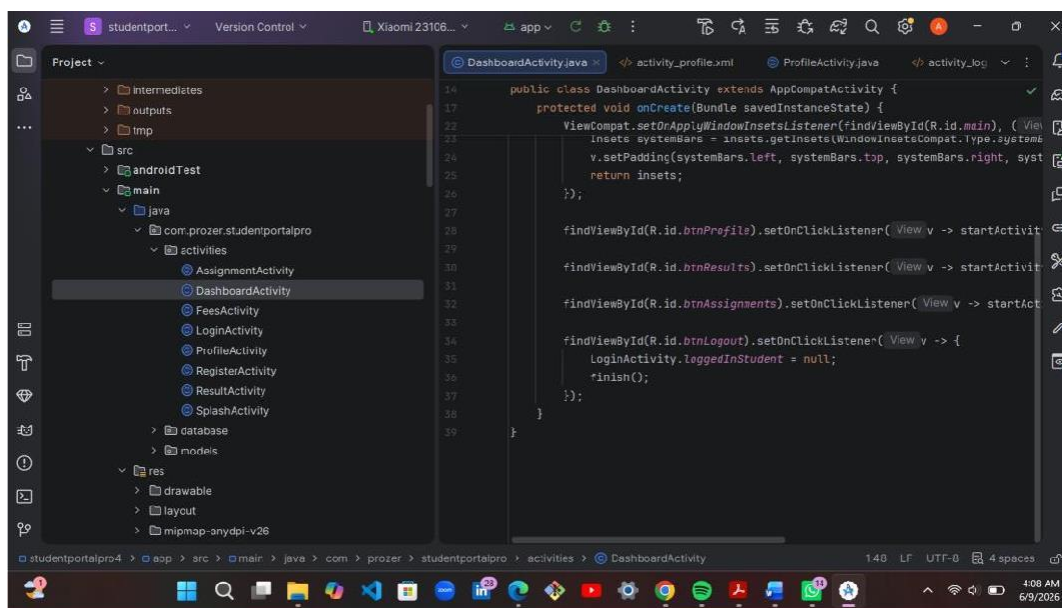
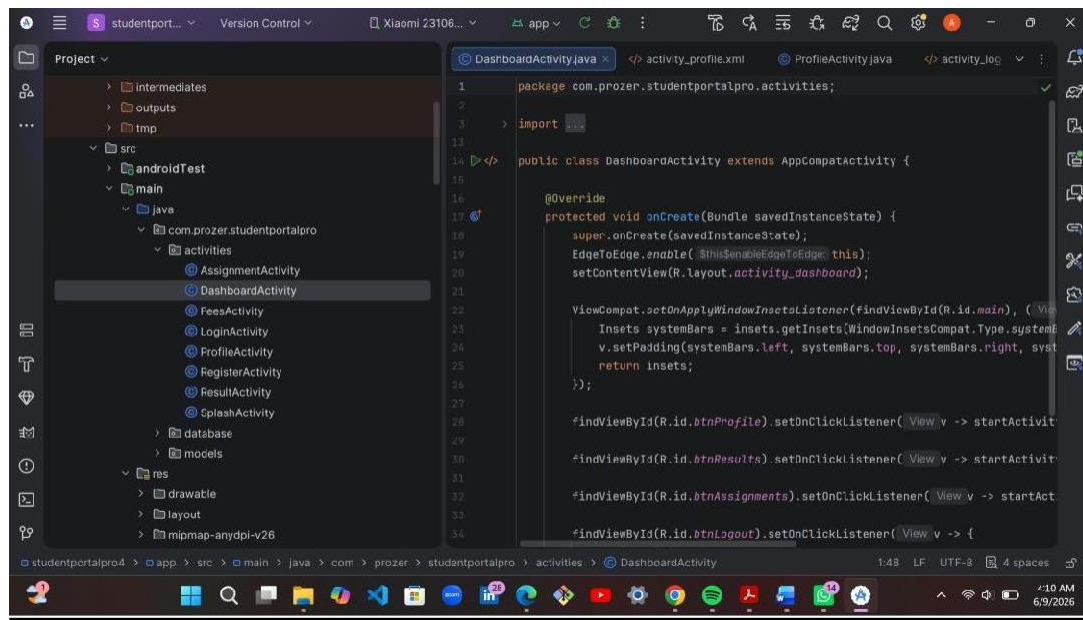
Figure 14.DASHBOARD

SCREENS&RESULTS BUTTON&RESULTS SCREENS



Figure

15.DASHBOARD SCREEN&ASSIGNMENT BUTTON&ASSIGNMENT SCREEN



Figure

16.JAVA CODE IMPLEMENTING BUTTON CLICK

PersonaReflection

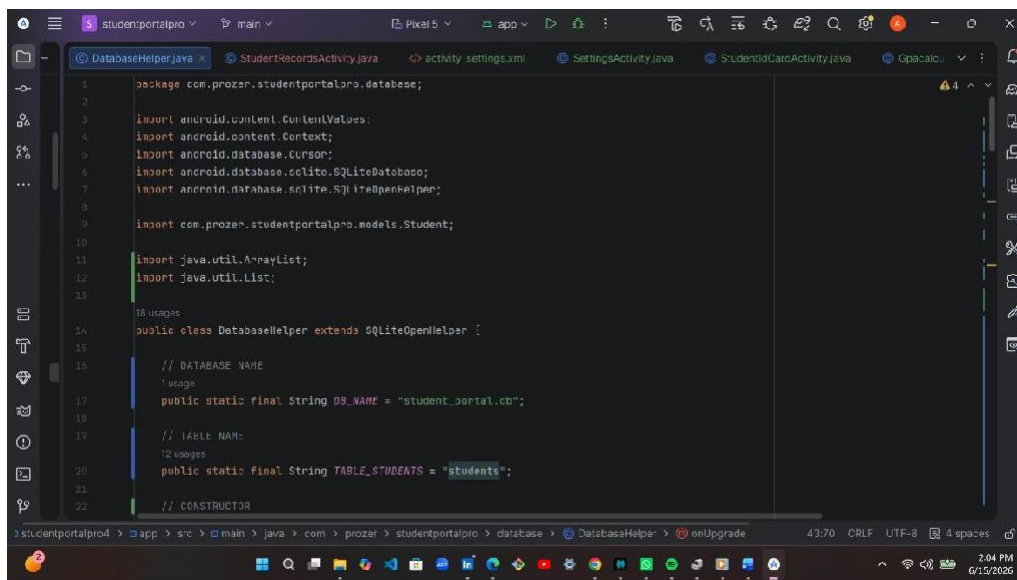
This week I learned how to connect multiple screens in an Android application. I also improved my understanding of navigation between activities and user interaction through buttons.

Week 4: Event Handling and User Interaction

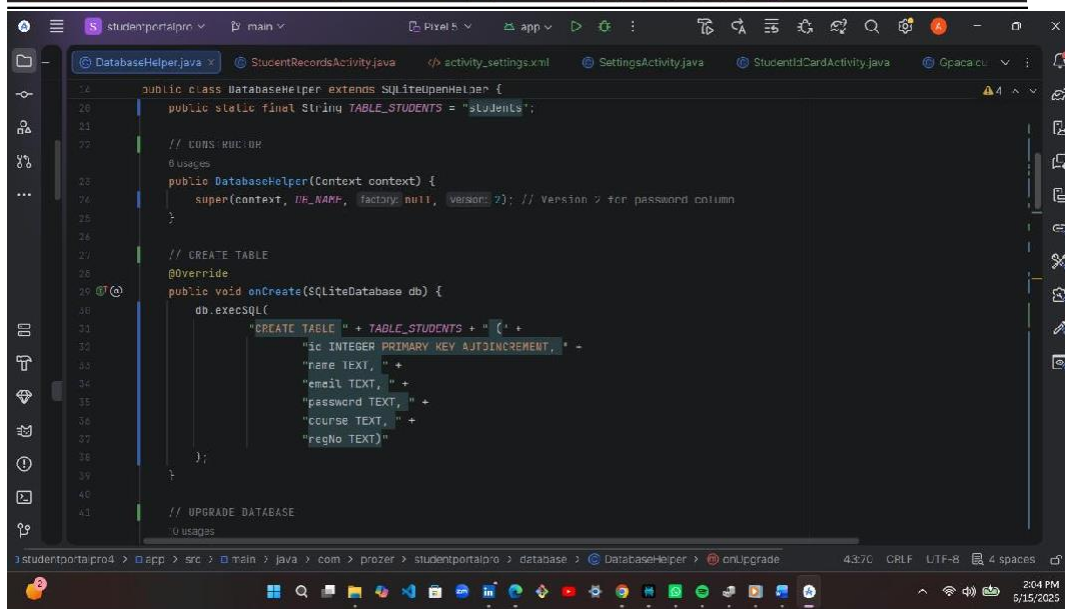
Practical Activities Undertaken

- Implemented SQLite database for student record management.
- Added Create, Read, Update and Delete (CRUD) operations.
- Developed RecyclerView for displaying student records.
- Implemented student search functionality.
- Added SharedPreferences for remembering login sessions.
- Improved dashboard with additional student services.

DatabaseHelper CRUD Methods



```
1 package com.prozer.studentportalpro.database;
2
3 import android.content.ContentValues;
4 import android.content.Context;
5 import android.database.Cursor;
6 import android.database.sqlite.SQLiteDatabase;
7 import android.database.sqlite.SQLiteOpenHelper;
8
9 import com.prozer.studentportalpro.models.Student;
10
11 import java.util.ArrayList;
12 import java.util.List;
13
14 // usage
15 public class DatabaseHelper extends SQLiteOpenHelper {
16
17     // DATABASE NAME
18     public static final String DB_NAME = "student_portal.db";
19
20     // TABLE NAME
21     public static final String TABLE_STUDENTS = "students";
22
23     // CONSTRUCTOR
```

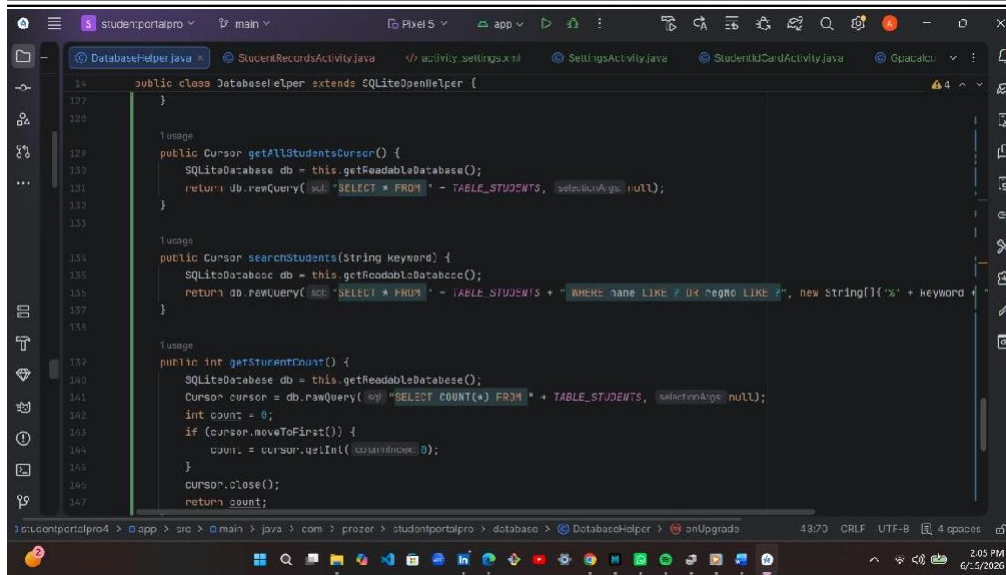


```
24 public DatabaseHelper(Context context) {
25     super(context, DB_NAME, factory: null, version: 2); // Version 2 for password column
26 }
27
28 // CREATE TABLE
29 @Override
30 public void onCreate(SQLiteDatabase db) {
31     db.execSQL(
32         "CREATE TABLE " + TABLE_STUDENTS + " (" +
33         "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
34         "name TEXT, " +
35         "email TEXT, " +
36         "password TEXT, " +
37         "course TEXT, " +
38         "regNo TEXT)"
39     );
40 }
41
42 // UPGRADE DATABASE
43 // usage
```

```
14 public class DatabaseHelper extends SQLiteOpenHelper {
41 // UPGRADE DATABASE
42 // @Override
43 @Override
44 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
45     db.execSQL("DROP TABLE IF EXISTS " + TABLE_STUDENTS);
46     onCreate(db);
47 }
48 // =====
49 // AUTHENTICATION
50 // =====
51
52 // Usage
53 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
54     SQLiteDatabase db = this.getWritableDatabase();
55     ContentValues values = new ContentValues();
56     values.put("name", name);
57     values.put("email", email);
58     values.put("password", password);
59     values.put("course", course);
60     values.put("regNo", regNo);
61
62     long result = db.insert(TABLE_STUDENTS, null, values);
63 }
```

```
14 public class DatabaseHelper extends SQLiteOpenHelper {
41 // UPGRADE DATABASE
42 // @Override
43 @Override
44 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
45     db.execSQL("DROP TABLE IF EXISTS " + TABLE_STUDENTS);
46     onCreate(db);
47 }
48 // =====
49 // AUTHENTICATION
50 // =====
51
52 // Usage
53 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
54     SQLiteDatabase db = this.getWritableDatabase();
55     ContentValues values = new ContentValues();
56     values.put("name", name);
57     values.put("email", email);
58     values.put("password", password);
59     values.put("course", course);
60     values.put("regNo", regNo);
61
62     long result = db.insert(TABLE_STUDENTS, null, values);
63 }
```

```
14 public class DatabaseHelper extends SQLiteOpenHelper {
52 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
53 }
54
55 // Usage
56 public Student loginStudent(String email, String password) {
57     SQLiteDatabase db = this.getWritableDatabase();
58     cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_STUDENTS + " WHERE email=? AND password=?", new String[] {email, password});
59
60     if (cursor != null && cursor.moveToFirst()) {
61         Student student = new Student(
62             cursor.getInt(cursor.getColumnIndex(0)),
63             cursor.getString(cursor.getColumnIndex(1)),
64             cursor.getString(cursor.getColumnIndex(2)),
65             cursor.getString(cursor.getColumnIndex(4)), // course
66             cursor.getString(cursor.getColumnIndex(5)) // regNo
67         );
68         cursor.close();
69         return student;
70     }
71     return null;
72 }
73
74 // Usage
75 public boolean updatePassword(int id, String newPassword) {
76 }
```



Registration Screen

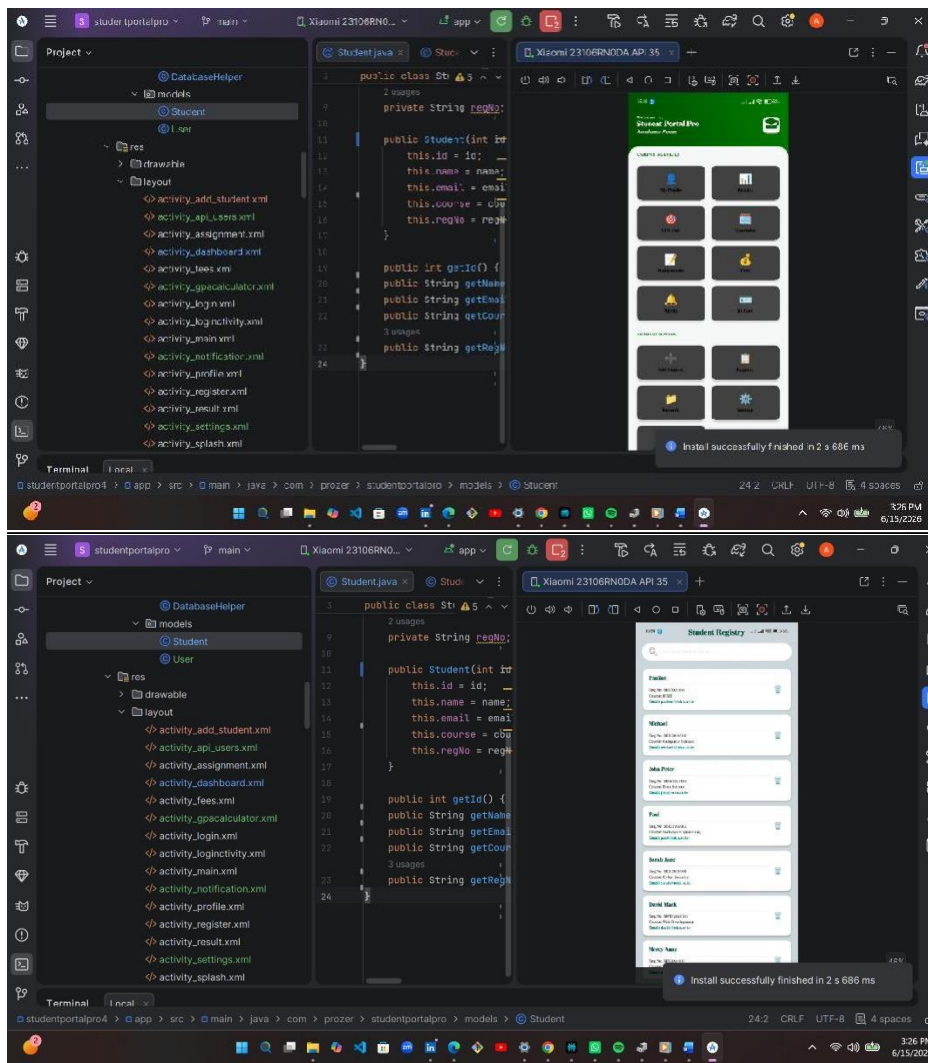


Figure 2. Student registration screen

ADD STUDENT

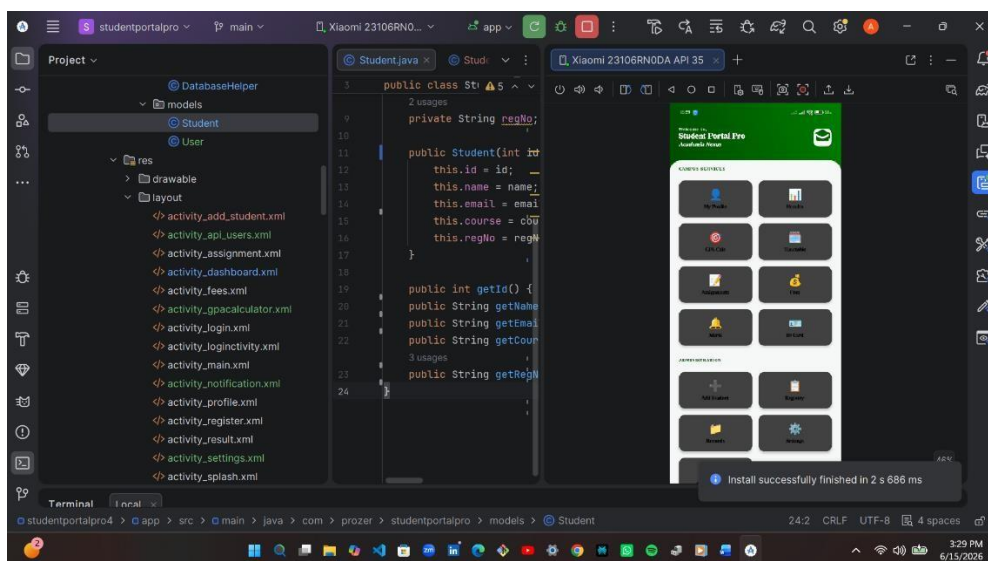


Figure 3. dashboard with add student

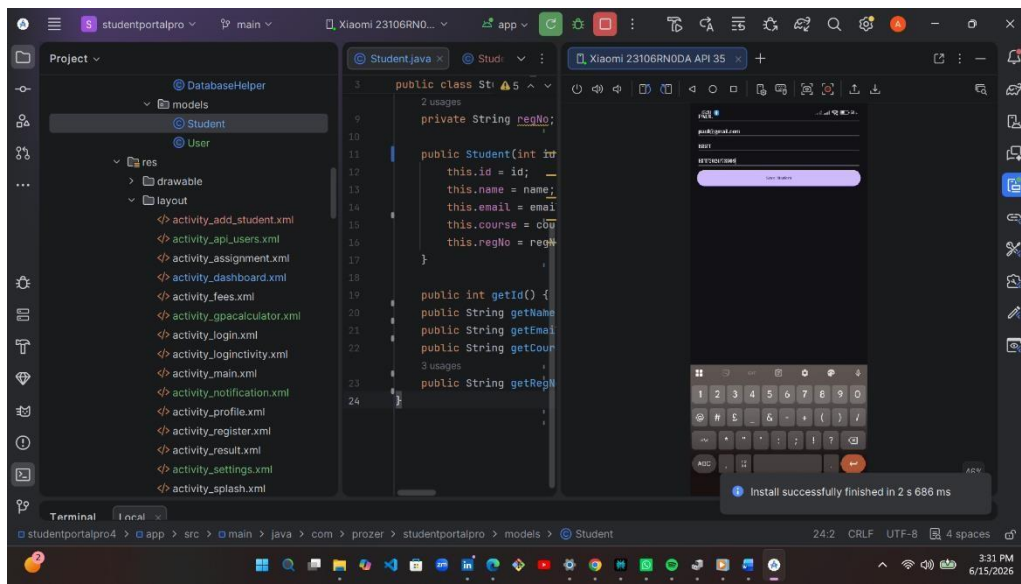


Figure 4.add student

button

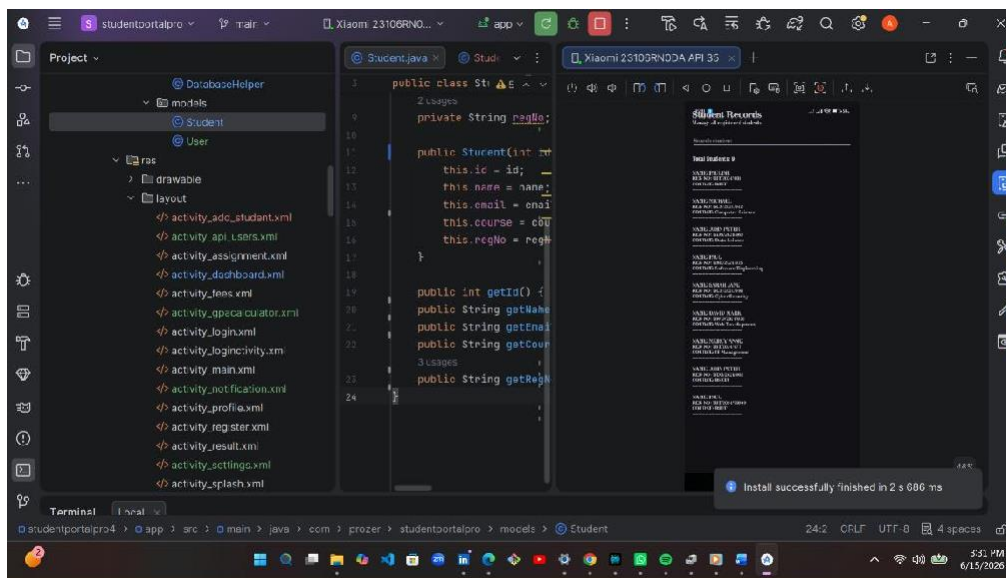


Figure 5.add student on student records

RecyclerView Student List

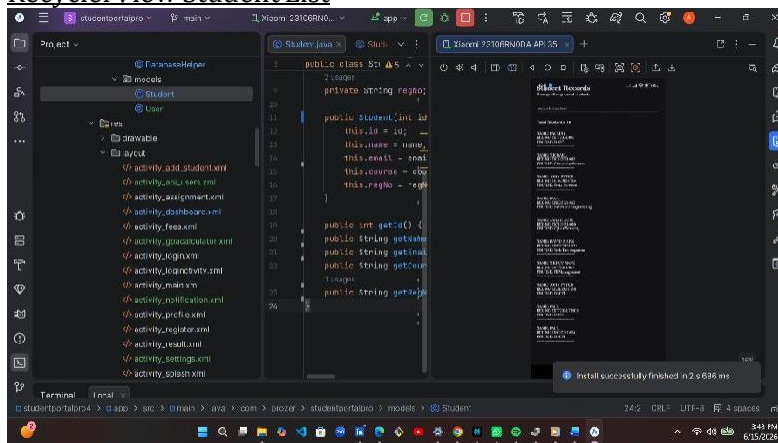


Figure 6.recycle view student list

Search Student

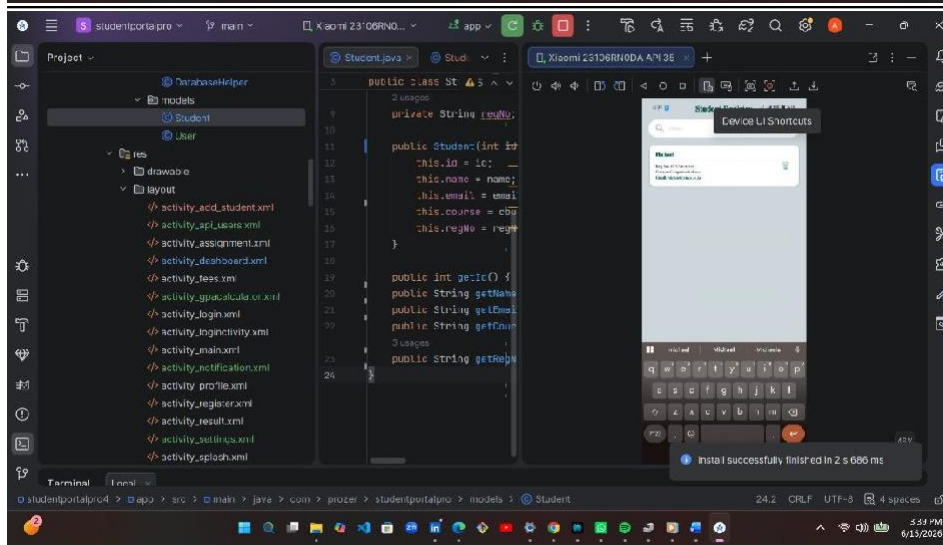
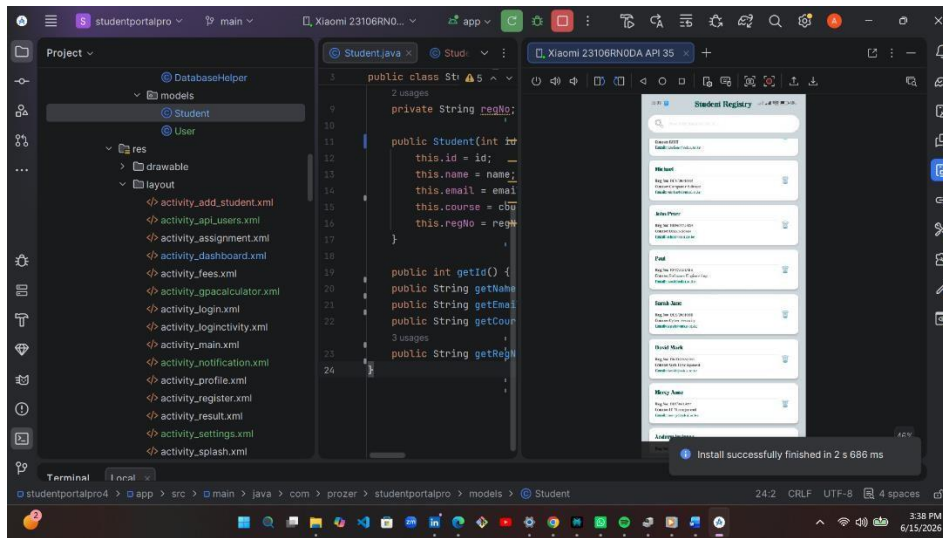
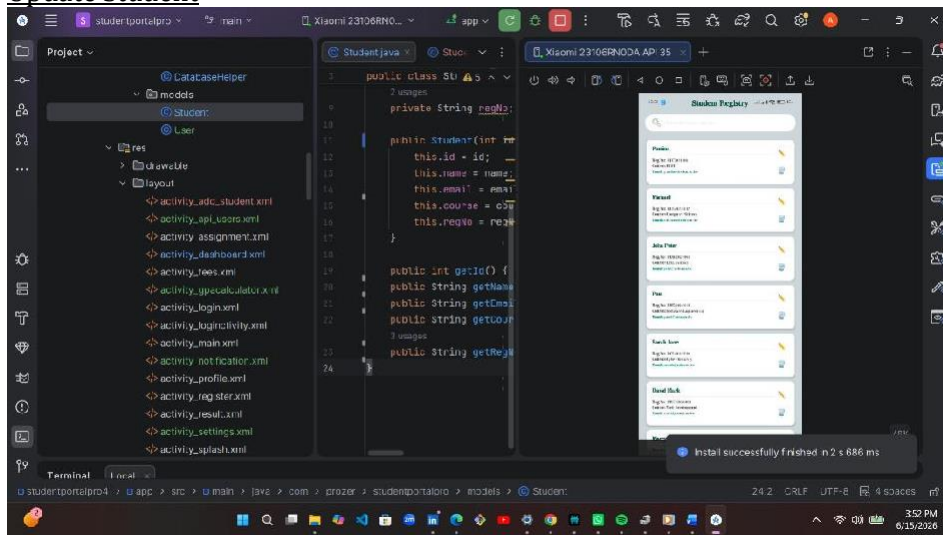


Figure 7. student search functionality

Update Student



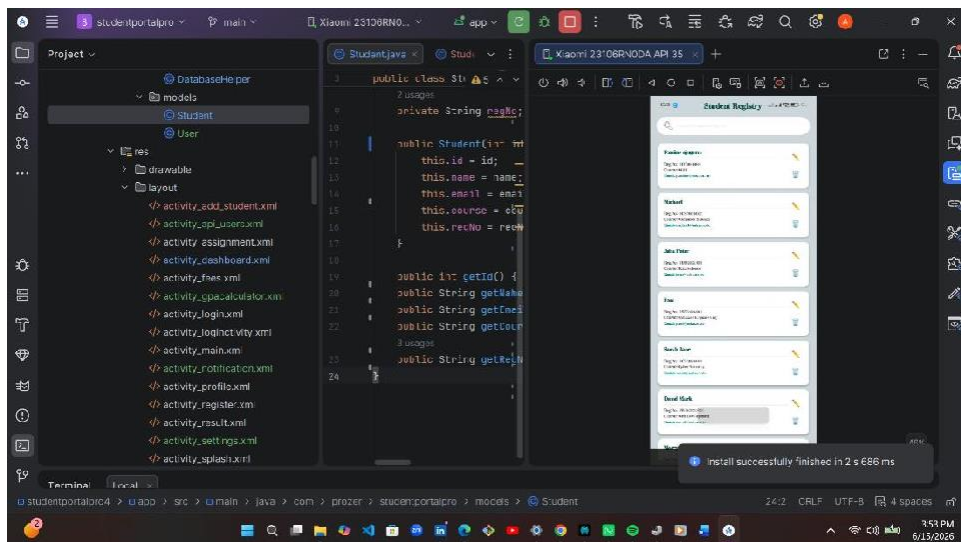
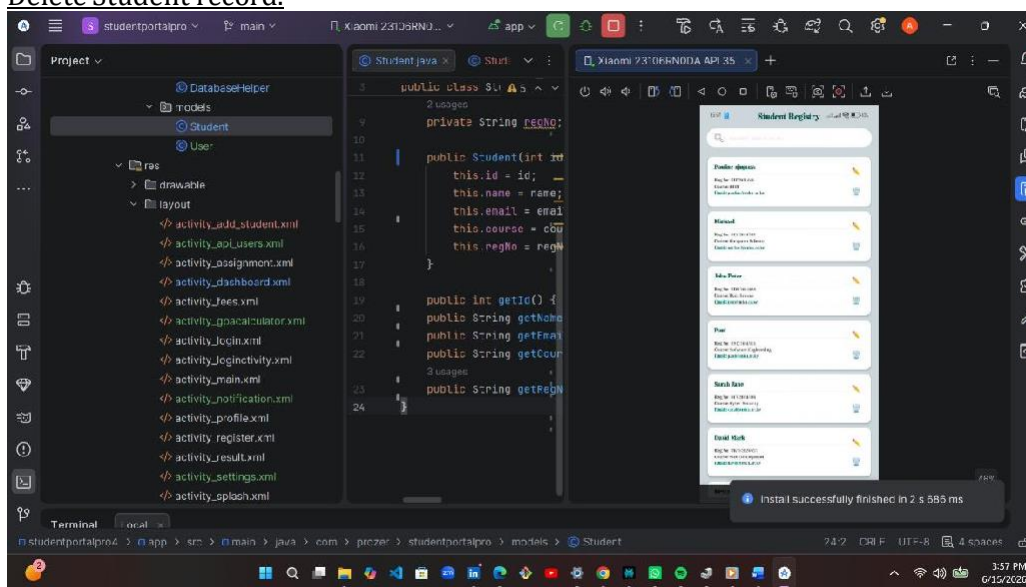


Figure 8 Updating Student Information

Delete Student record.



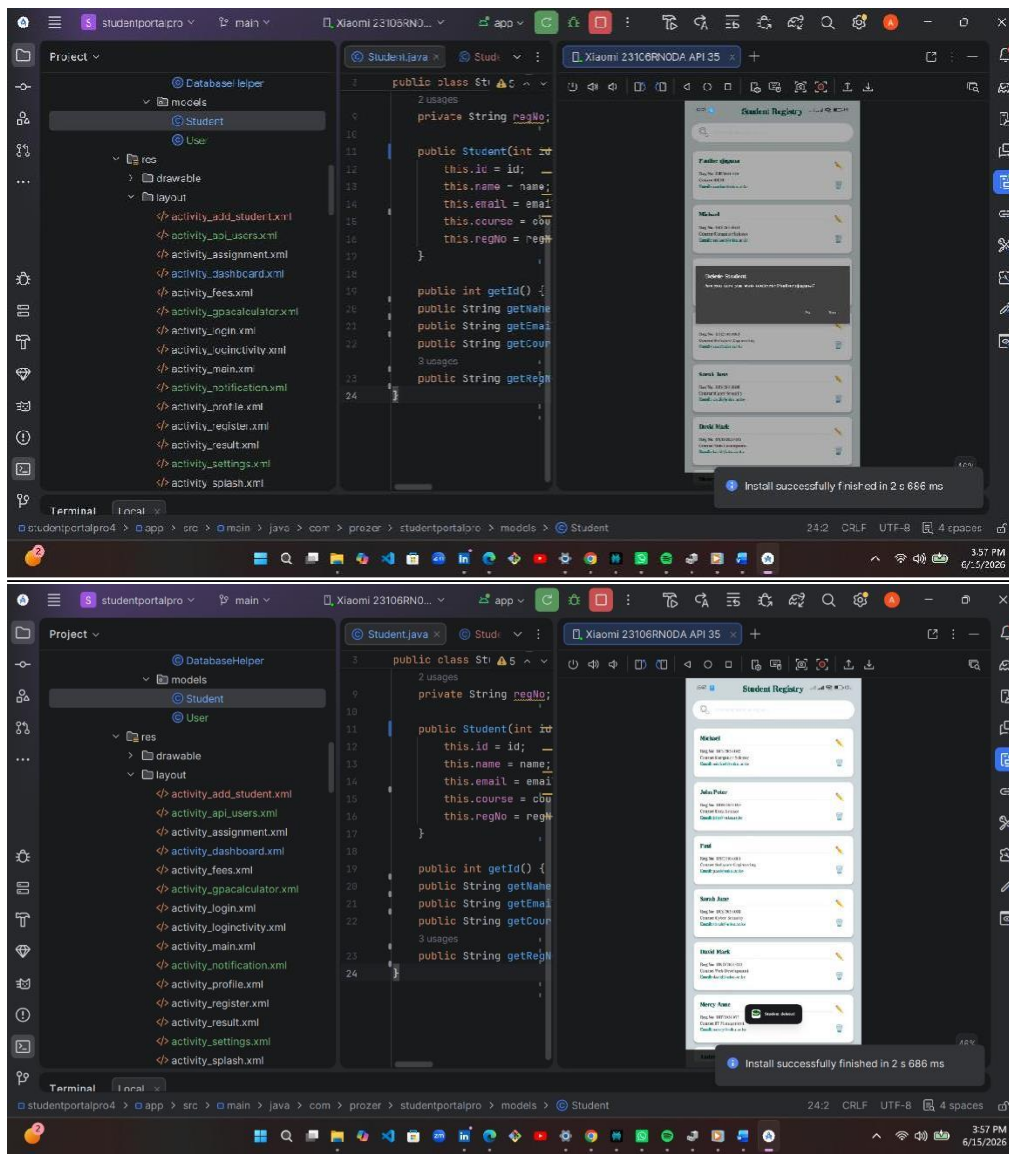


Figure 9 Student Record Deletion

Enhanced Student Portal Dashboard

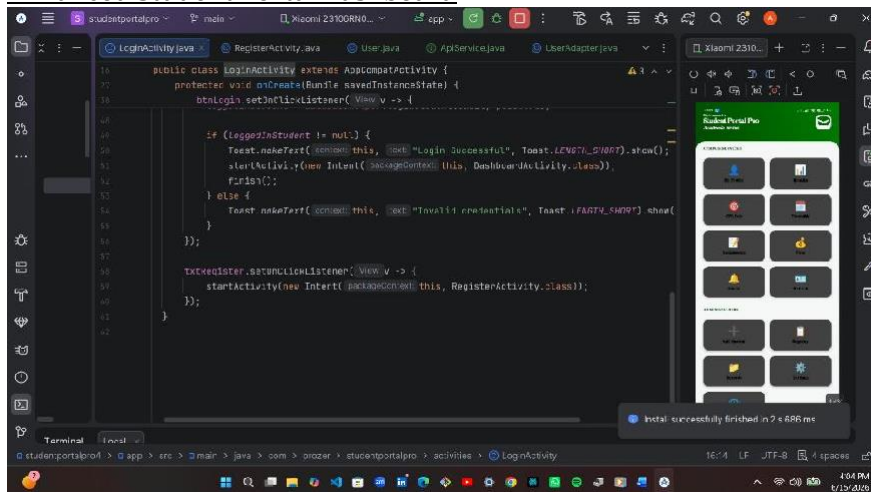


Figure 10 Enhanced Student Portal Dashboard

TOOLS/SOFTWARE USED

1. Android Studio

2. Java
3. SQLite Database
4. Recycler View
5. Shared Preferences
6. Android Emulator

PERSONAL REFLECTION

This week I learned how to manage data using SQLite database in Android applications. I implemented CRUD operations, Recycler View, search functionality, and Shared Preferences, which improved my understanding of data management and user interaction.

Week 5: Data Management

NewsActivity Created

PRACTICAL ACTIVITIES

- Implemented networking functionality using Retrofit.
- Created model classes for handling news data.
- Developed API service interfaces for HTTP requests.
- Configured Retrofit client for server communication.
- Parsed JSON responses and displayed data using RecyclerView.
- Added ProgressBar to indicate loading state.
- Implemented Internet permission and error handling.
- Integrated Campus News feature into the Student Portal Pro application

NewsActivity Created

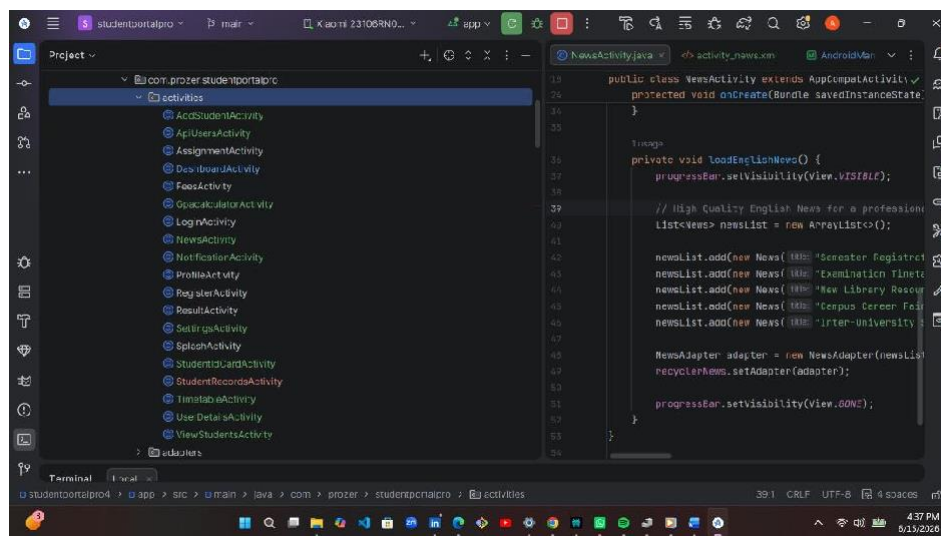


Figure 11.CREATION OF NEWS OF ACTIVITY

News Layout Created

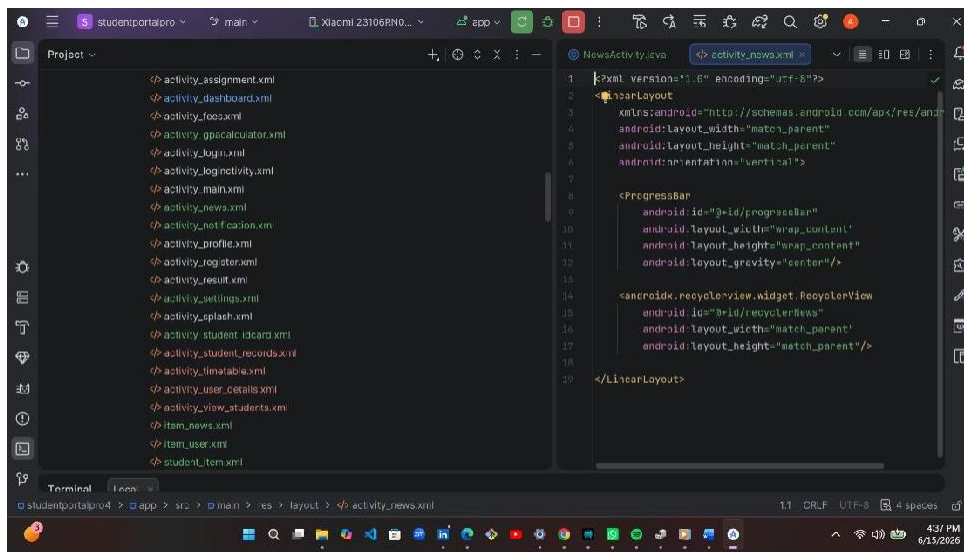


Figure 12 NEWS LAYOUT CREATED

Internet Permission

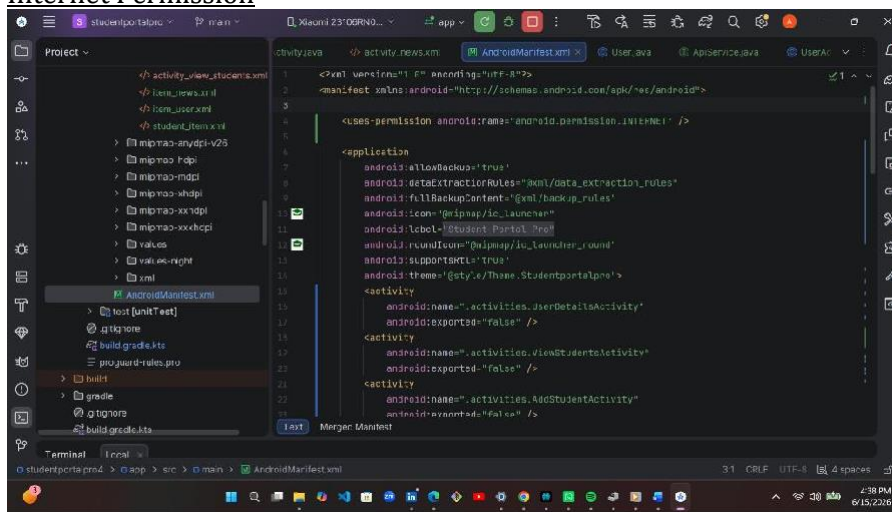


Figure 13.INTERNET PERMISSION

Dashboard Button

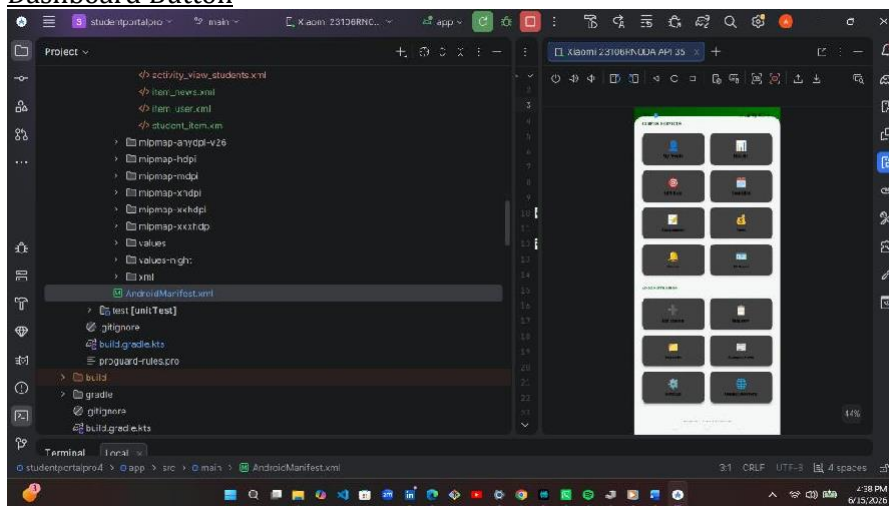


Figure 14.DASHBOARD BUTTON OF CAMPUS NEWS

Campus News Opens

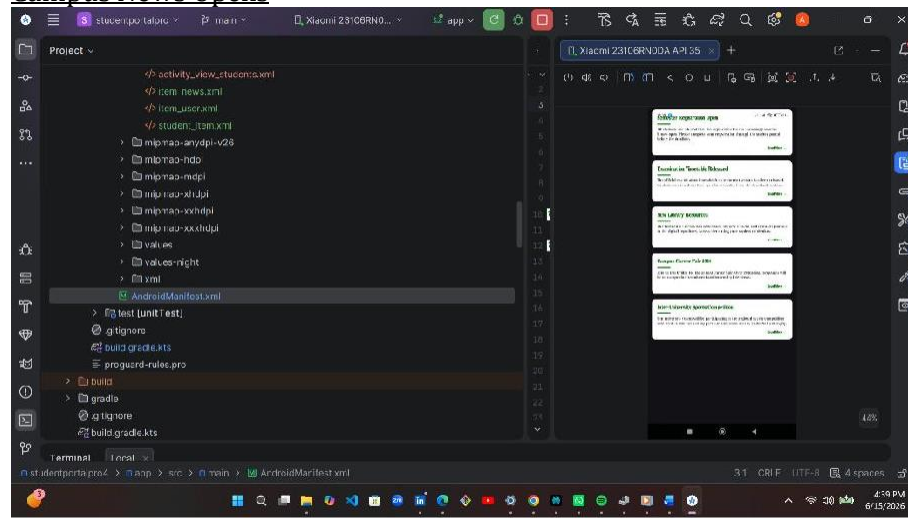
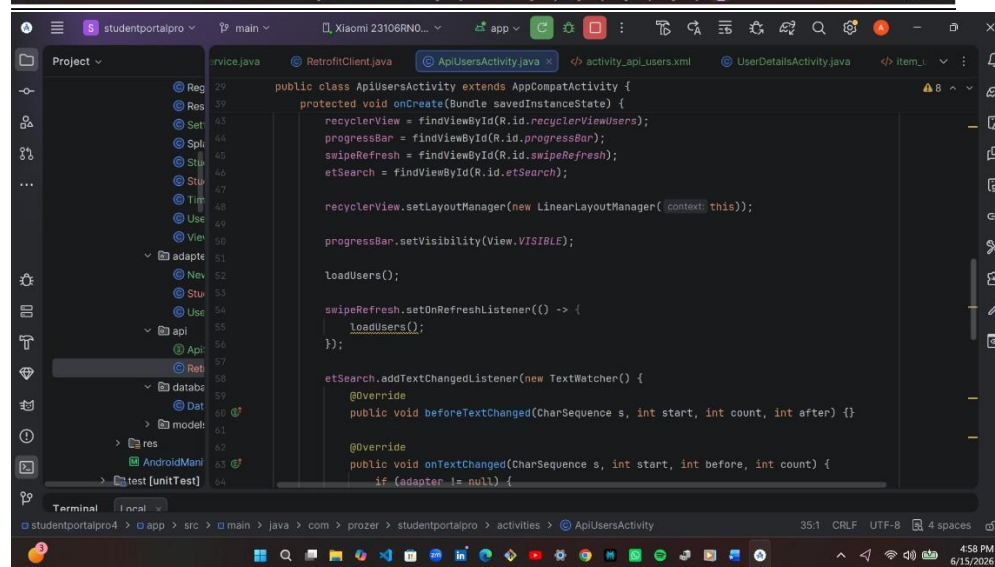
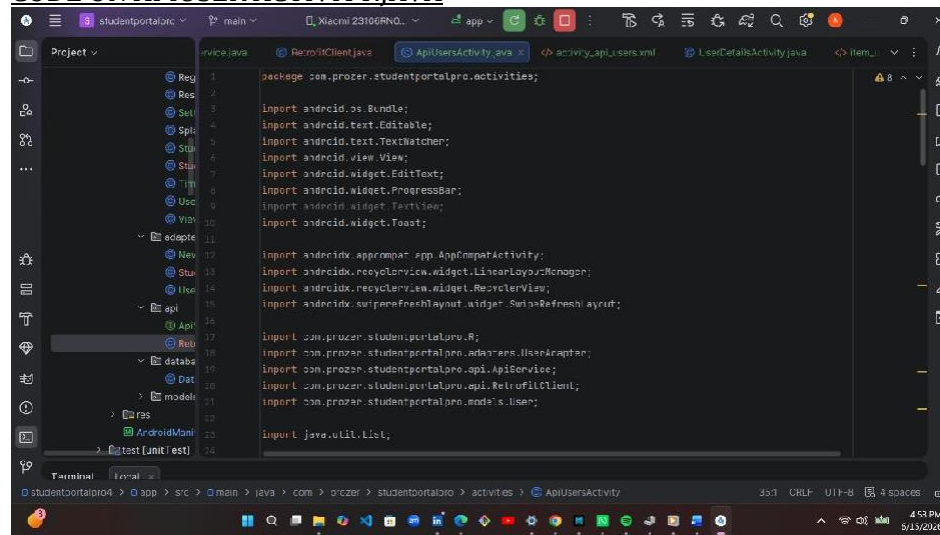


Figure 15NEWS SCREEN INTERFACE

API Service Interface CODE ON APIUSER ACTIVITY.JAVA



```

public class APIUsersActivity extends AppCompatActivity {
    private void loadUsers() {
        ApiService.getUsers().enqueue(new CallBack<List<User>>() {
            public void onResponse(Call<List<User>> call, Response<List<User>> response) {
                RecyclerView.setAdapter(adapter);
            }
        });
    }

    @Override
    public void onFailure(Call<List<User>> call, Throwable t) {
        progressBar.setVisibility(View.GONE);
        swipeRefreshLayout.setRefreshing(false);
        Toast.makeText(APIUsersActivity.this, t.getMessage(), Toast.LENGTH_SHORT);
    }
}

```

Figure 16APIUSERACTIVITY.JAVA

Retrofit Client Configuration

```

package com.prozox.studentportalpro.api;

import ...

public class RetrofitClient {

    private static final String BASE_URL =
        "https://jsonplaceholder.typicode.com/";

    private static Retrofit retrofit;

    public static Retrofit getRetrofitInstance() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(
                    GsonConverterFactory.create()
                )
            }
    }
}

```

```

public class RetrofitClient {

    private static final String BASE_URL =
        "https://jsonplaceholder.typicode.com/";

    private static Retrofit retrofit;

    public static Retrofit getRetrofitInstance() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(
                    GsonConverterFactory.create()
                )
                .build();
        }
        return retrofit;
    }
}

```

Figure 17RETROIT CLIENT CONFIGURATION

TOOLS/SOFTWARE USED

Android Studio

Java

Retrofit

RecyclerView

JSON

REST API

Android Emulator

Android SDK

PERSONAL REFLECTION

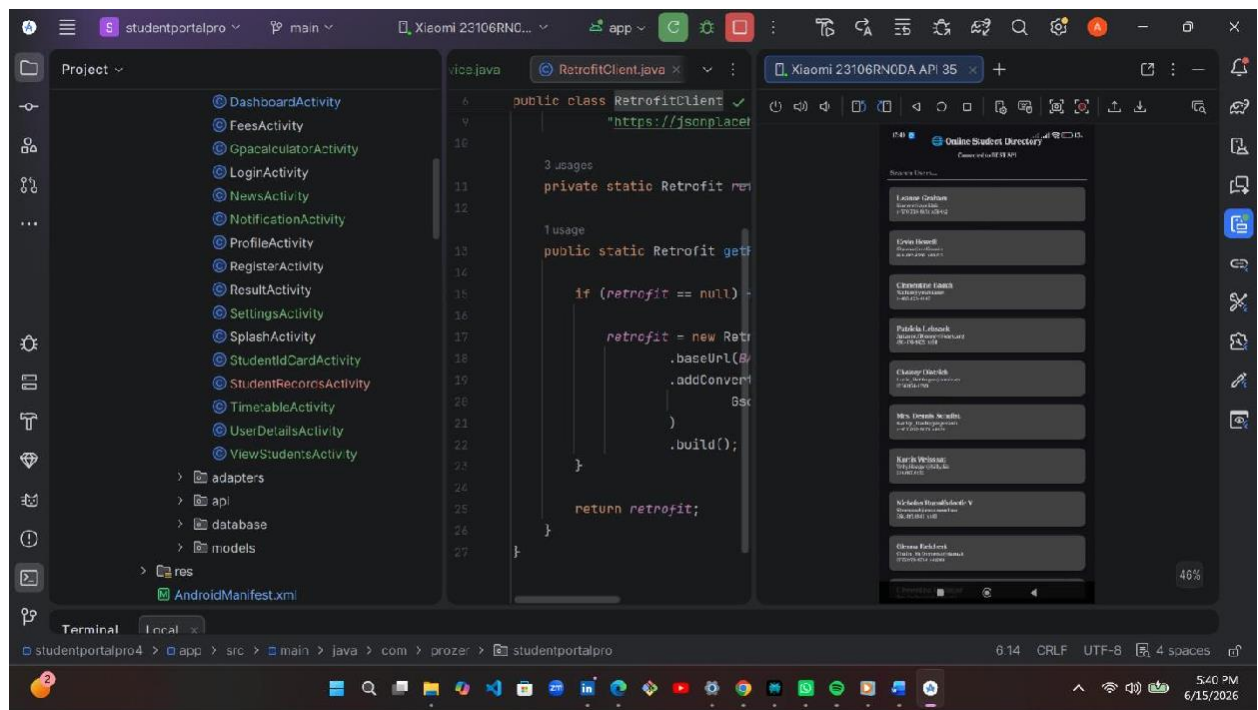
This week I learned how Android applications communicate with remote servers through APIs. I implemented Retrofit networking, JSON parsing, RecyclerView data presentation, and loading indicators. These activities improved my understanding of modern mobile application development and real-time data retrieval.

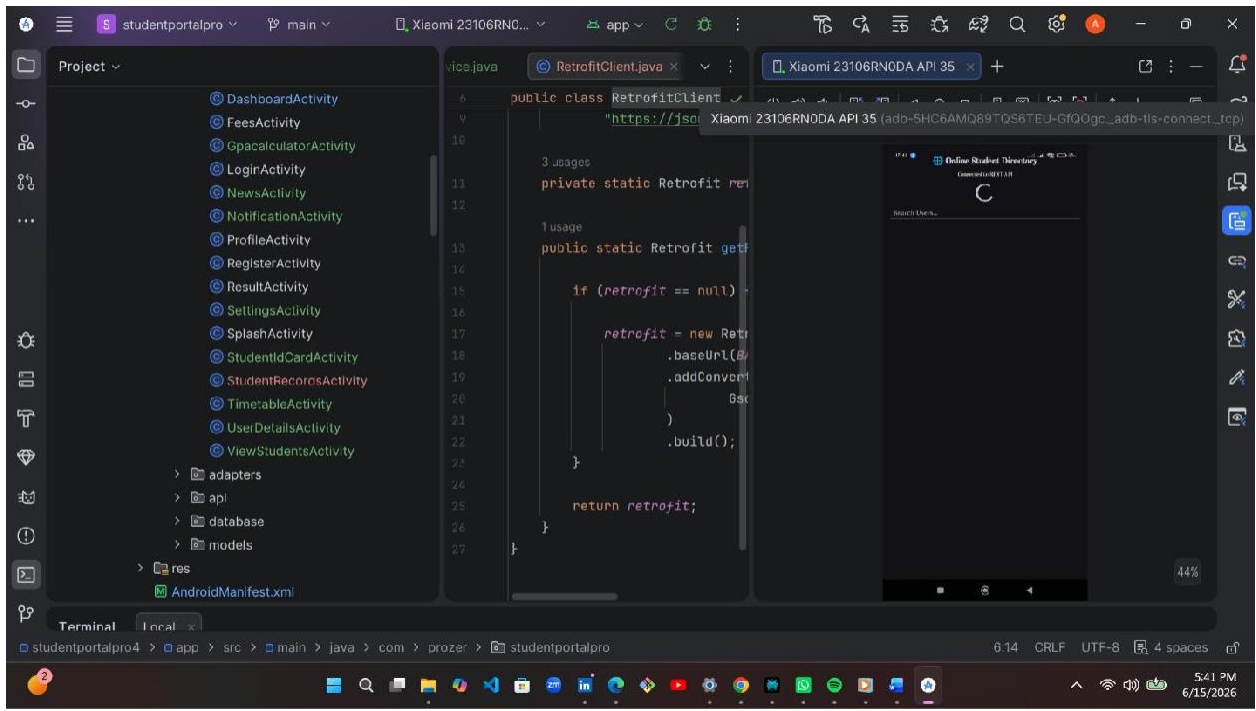
Week 6: Networking and APIs

PRACTICAL ACTIVITIES UNDERTAKEN

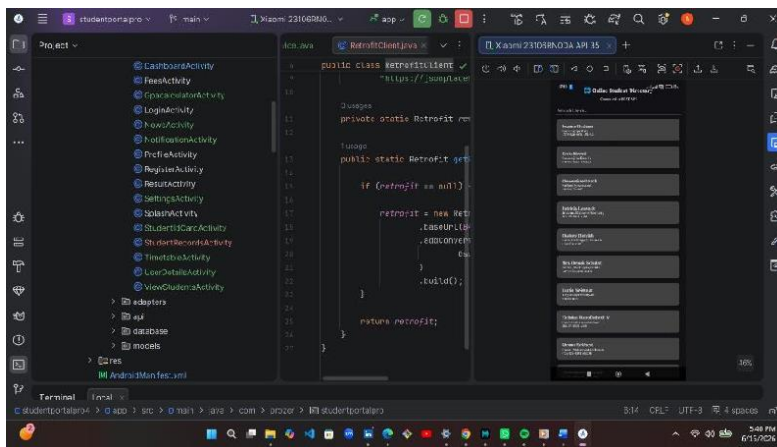
- Implemented an Online Directory module using API communication.
- Retrieved user information from a remote server.
- Displayed fetched data using Recycler View.
- Created model and adapter classes for user management.
- Tested successful network communication and data display.
- Integrated the Online Directory into the Student Portal Pro dashboard.

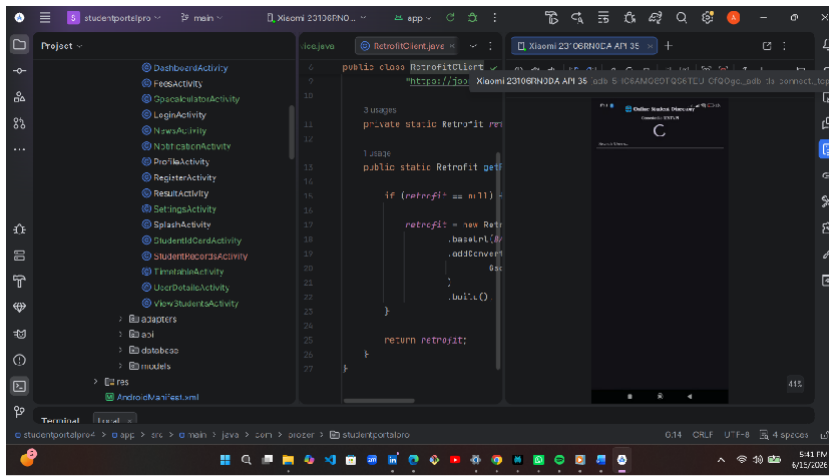
Online Directory Activity Code



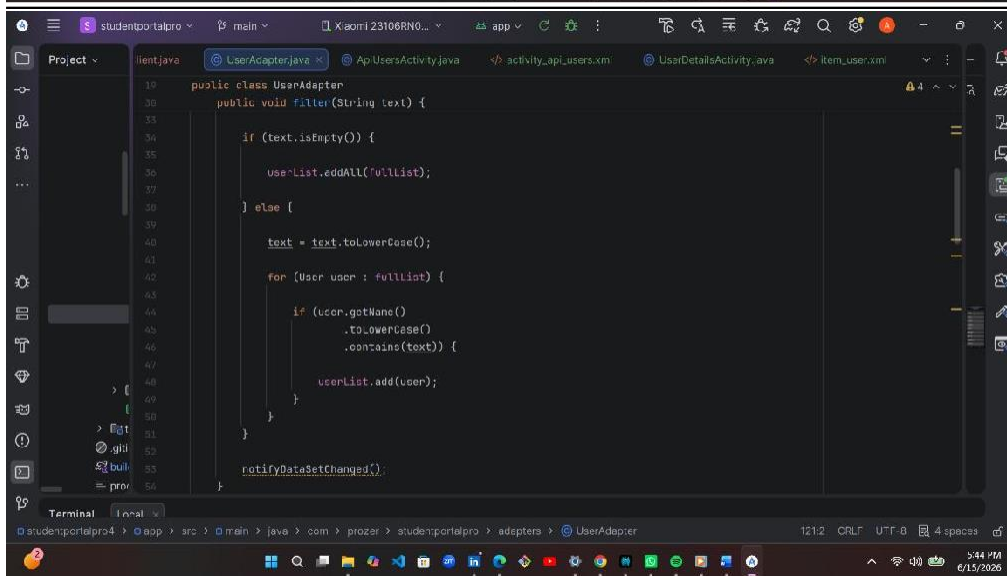
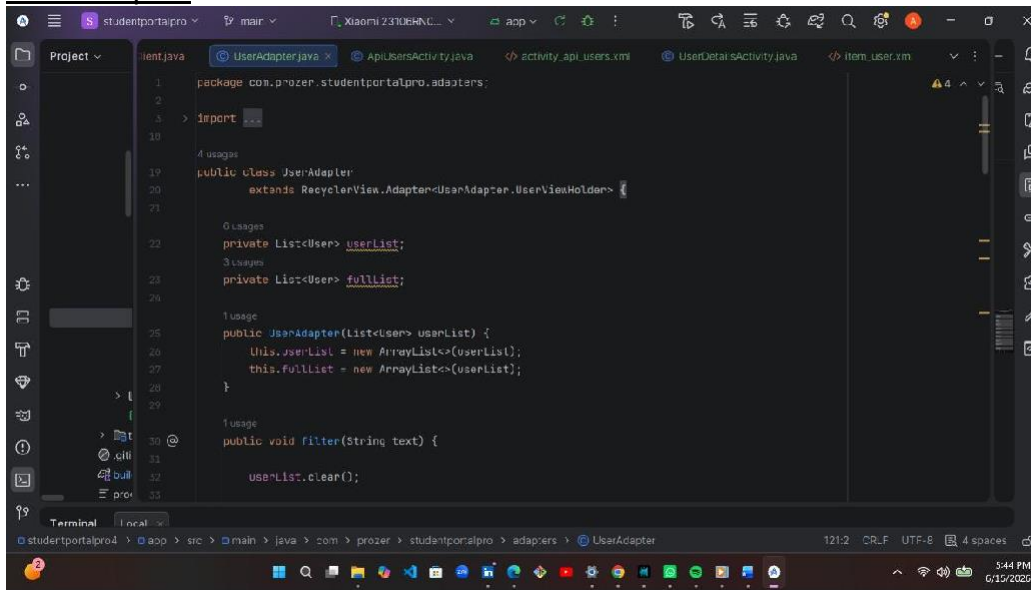


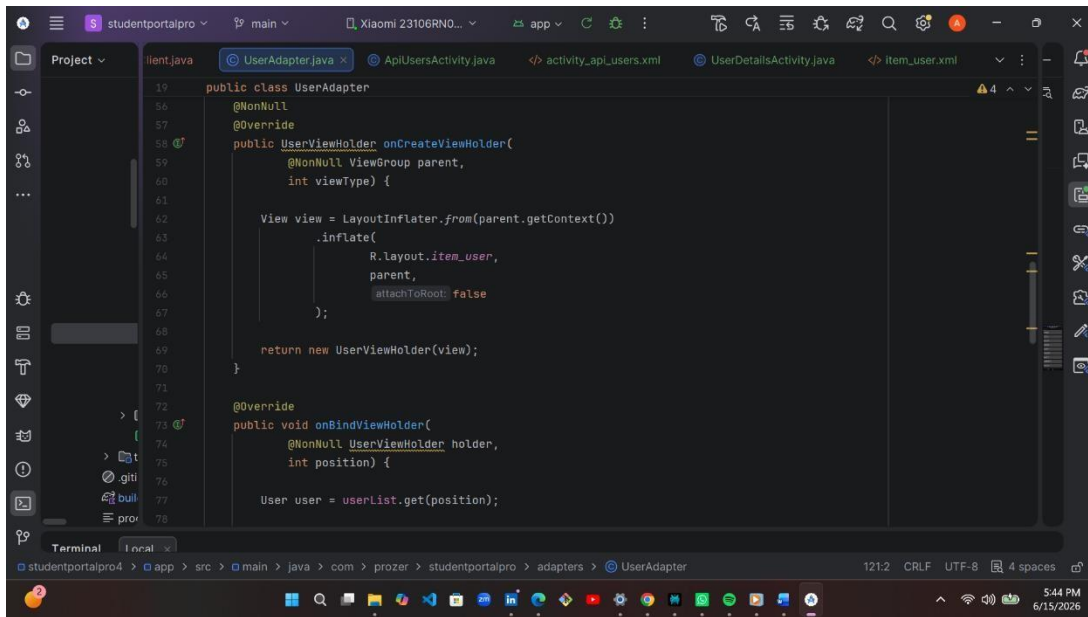
API Call INSIDE ONLINE DIRECTORY





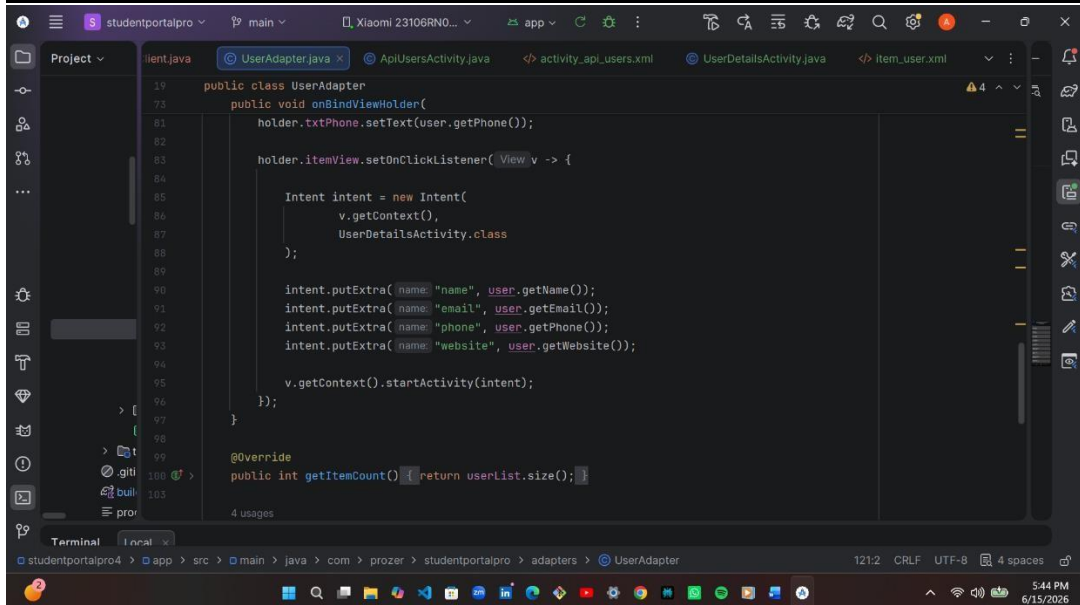
User Adapter





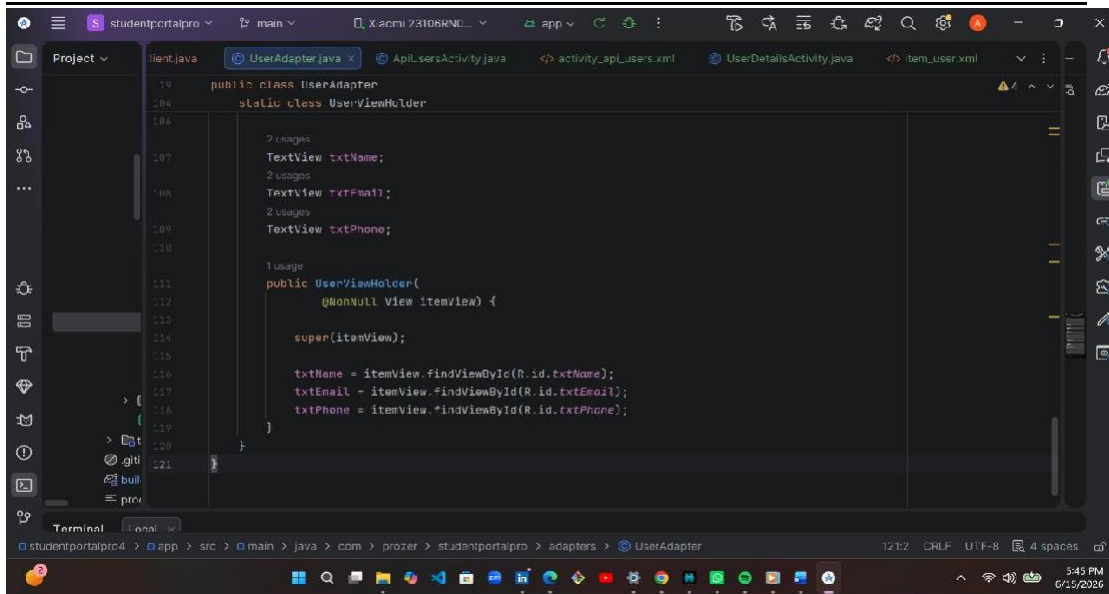
This screenshot shows the `onCreateViewHolder` method of the `UserAdapter` class. The method inflates the `item_user.xml` layout and returns a new `UserViewHolder` object. The `ViewHolder` class is not yet defined in this view.

```
19 public class UserAdapter
56     @NonNull
57     @Override
58     public UserViewHolder onCreateViewHolder(
59         @NonNull ViewGroup parent,
60         int viewType) {
61
62         View view = LayoutInflater.from(parent.getContext())
63             .inflate(
64                 R.layout.item_user,
65                 parent,
66                 attachToRoot: false
67             );
68
69         return new UserViewHolder(view);
70     }
71
72     @Override
73     public void onBindViewHolder(
74         @NonNull UserViewHolder holder,
75         int position) {
76
77         User user = userList.get(position);
78     }
```



This screenshot shows the `onBindViewHolder` method of the `UserAdapter` class. It sets the text of the `txtPhone` field and sets an `onClick` listener on the `itemView`. The listener creates an `Intent` to launch `UserDetailsActivity` with user data as extras.

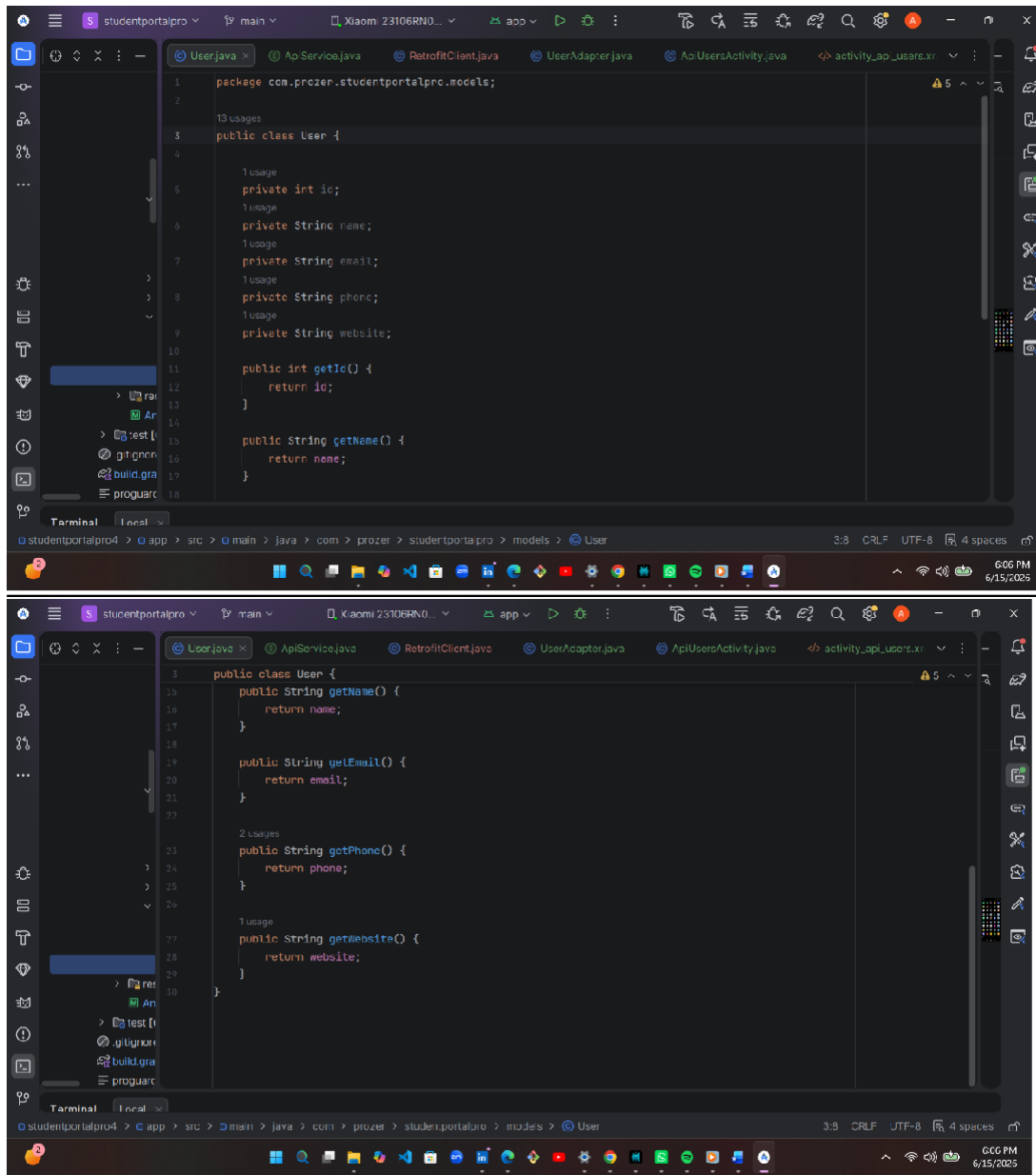
```
73 public void onBindViewHolder(
81     holder.txtPhone.setText(user.getPhone());
82
83     holder.itemView.setOnClickListener( View v -> {
84
85         Intent intent = new Intent(
86             v.getContext(),
87             UserDetailsActivity.class
88         );
89
90         intent.putExtra( name: "name", user.getName());
91         intent.putExtra( name: "email", user.getEmail());
92         intent.putExtra( name: "phone", user.getPhone());
93         intent.putExtra( name: "website", user.getWebsite());
94
95         v.getContext().startActivity(intent);
96     });
97
98
99     @Override
100     public int getItemCount() { return userList.size(); }
101 }
```



This screenshot shows the `UserViewHolder` class, which is a static inner class of `UserAdapter`. It defines the fields for the user data (name, email, phone) and implements the `onBindViewHolder` method to bind the data to the views.

```
19 public class UserAdapter
204     static class UserViewHolder
206
207         2 usages
208         TextView txtName;
209         2 usages
210         TextView txtEmail;
211         2 usages
212         TextView txtPhone;
213
214         1 usage
215         public UserViewHolder(
216             @NonNull View itemView) {
217
218             super(itemView);
219
220             txtName = itemView.findViewById(R.id.txtName);
221             txtEmail = itemView.findViewById(R.id.txtEmail);
222             txtPhone = itemView.findViewById(R.id.txtPhone);
223         }
224     }
```

User Model



TOOLS USED

- Android Studio
- Java
- Retrofit

- Recycler View
- REST API
- JSON
- Android Emulator

PERSONAL REFLECTION

This week I implemented an Online Directory feature that retrieves user information from a remote server through APIs. I learned how network requests are made, how data is processed, and how information can be displayed dynamically in Android applications. This improved my understanding of networking concepts and API integration.

Week 7: Application Architecture

Practical Activities Undertaken

- Implemented networking functionality using Retrofit.
- Connected the application to a public REST API.
- Retrieved user information from a remote server.
- Displayed API data using RecyclerView.
- Implemented user search functionality.
- Tested network communication and error handling.
- Integrated the Online Directory feature into the Student Portal Pro dashboard.

Add Student Interface

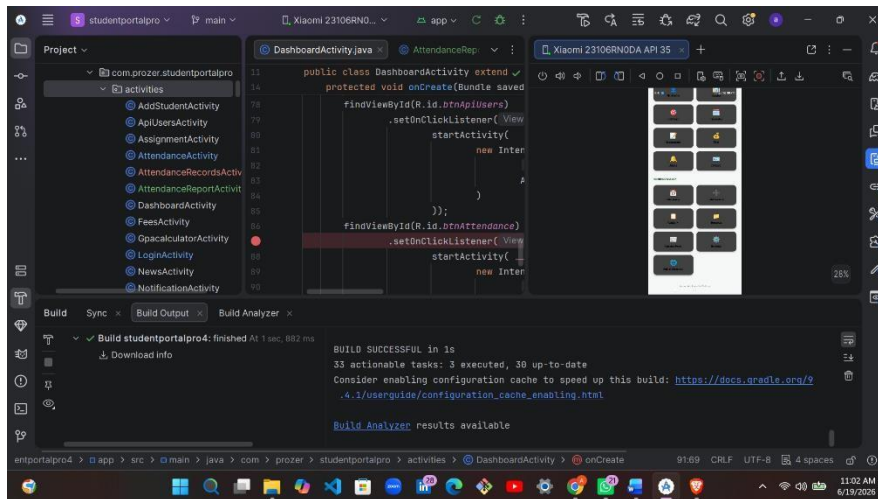


Figure 18.dashboard with add

student screen

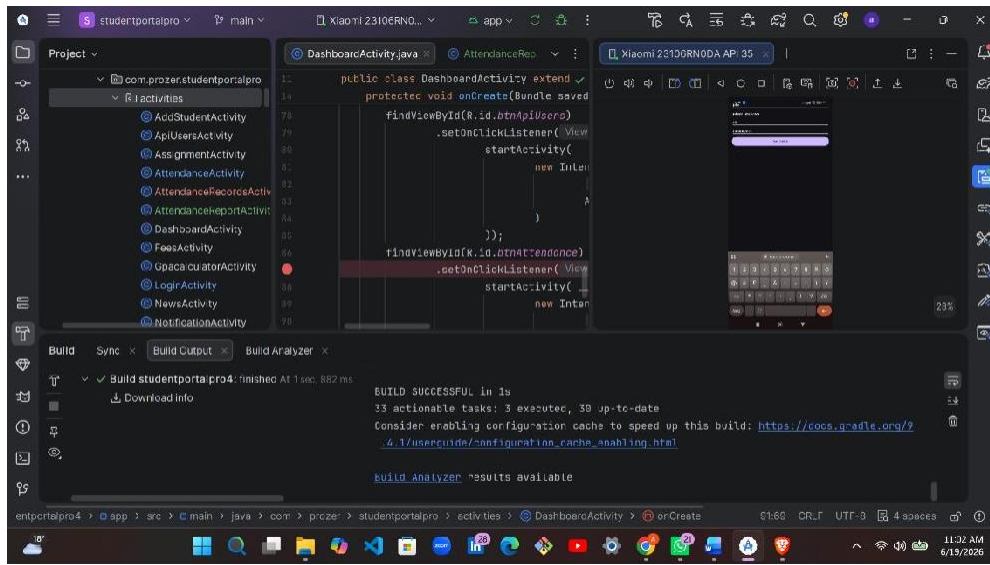


Figure 19 inserting the names on the add students screen

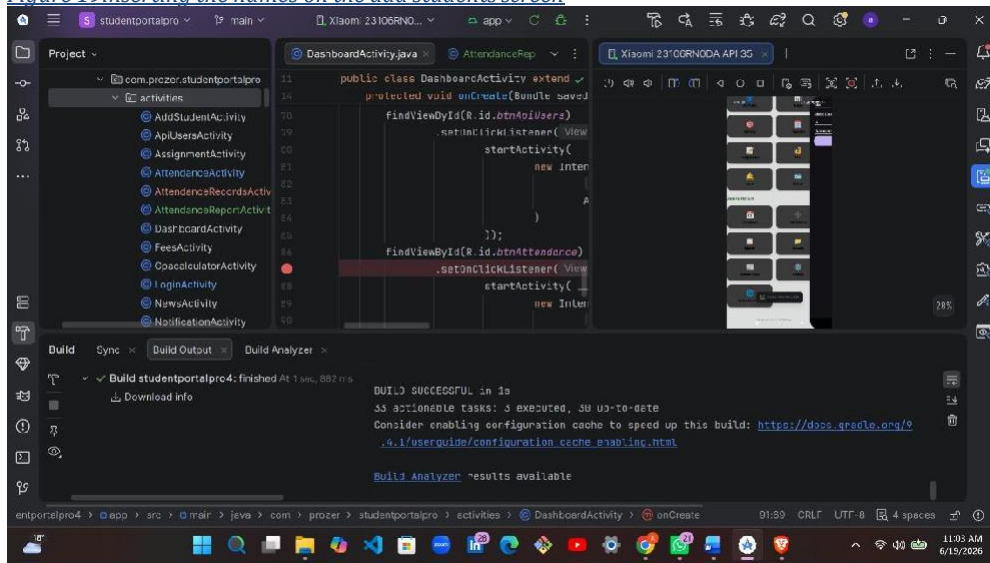


Figure 20.the student details saved

Student Records Display

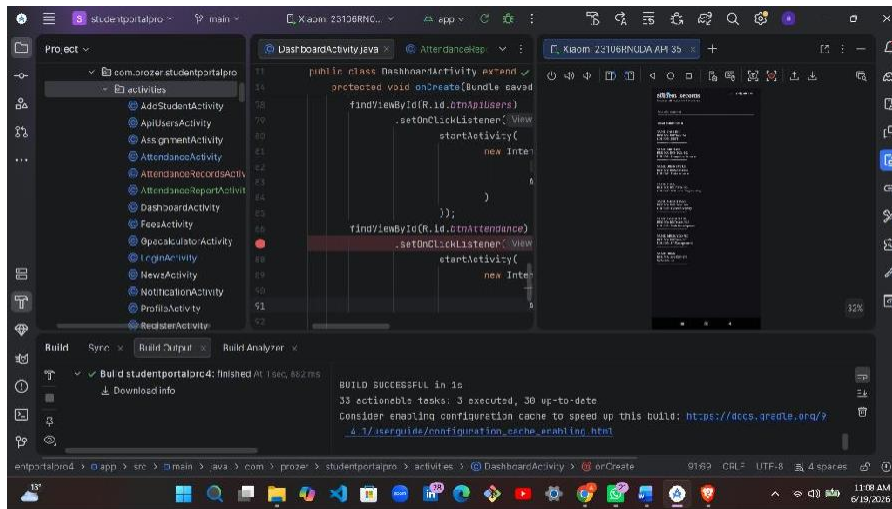


Figure 21.list of students

Student Search Function

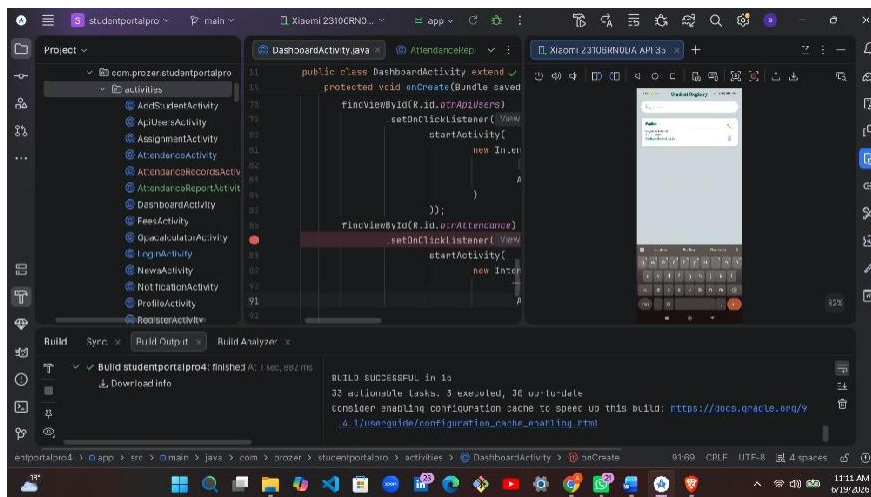


Figure 22.search results

Attendance Registration Module

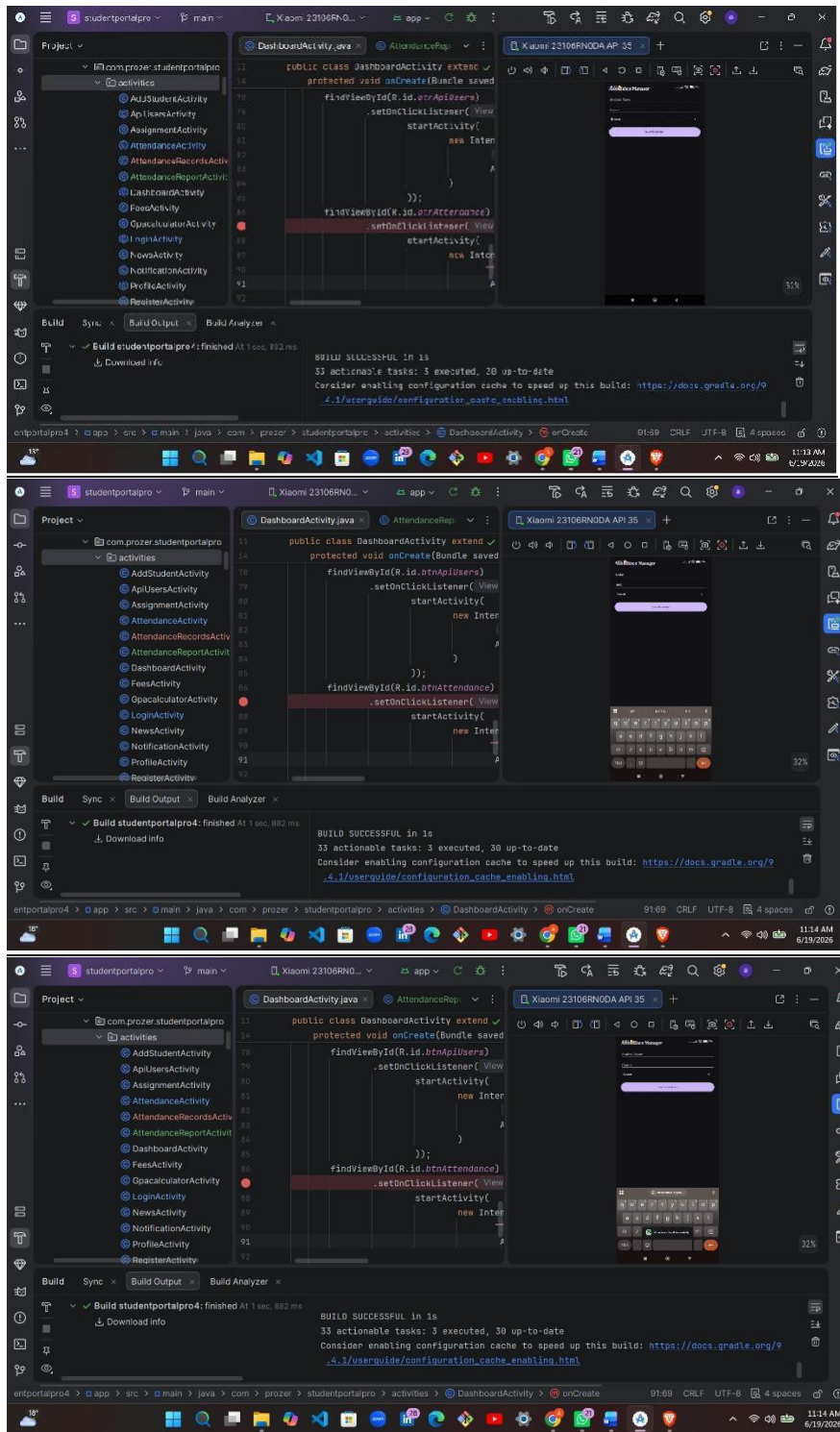


Figure 23. ATTENDANCE SCREEN BEFORE AND AFTER SAVING

Login Persistence (Shared Preferences)

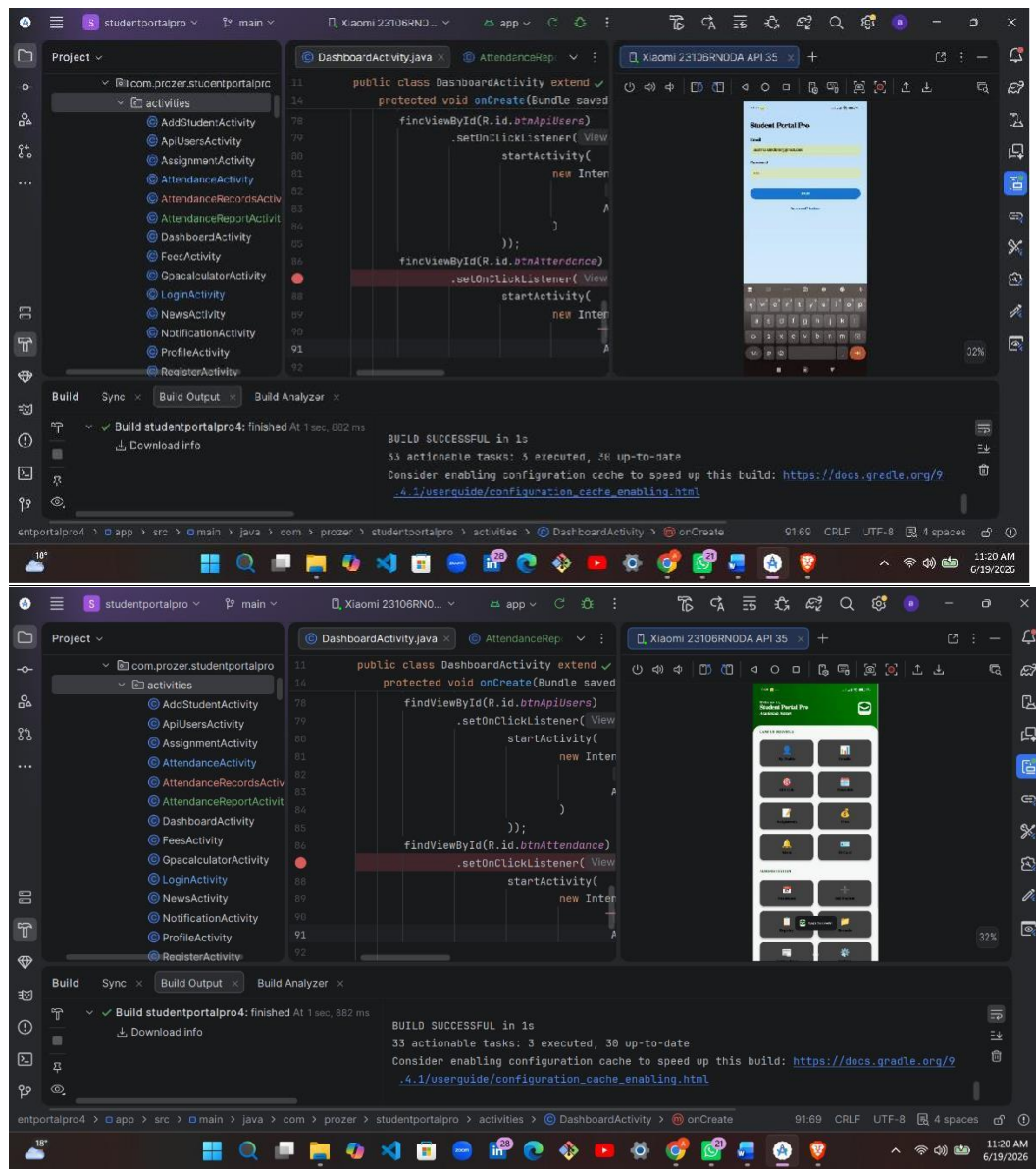


Figure 24 Shared Preferences Session Persistence

TOOLS USED

1. Android Studio
2. Java
3. SQLite Database
4. Shared Preferences
5. XML
6. Android SDK
7. Android Emulator

PERSONAL REFLECTION

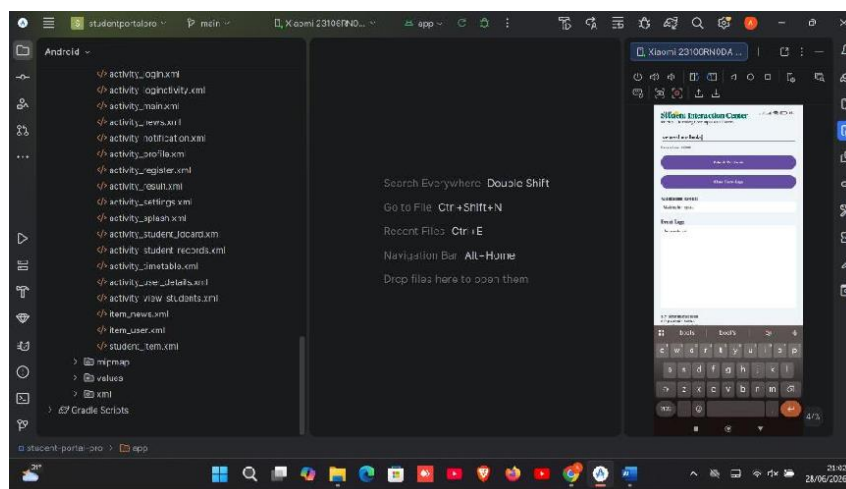
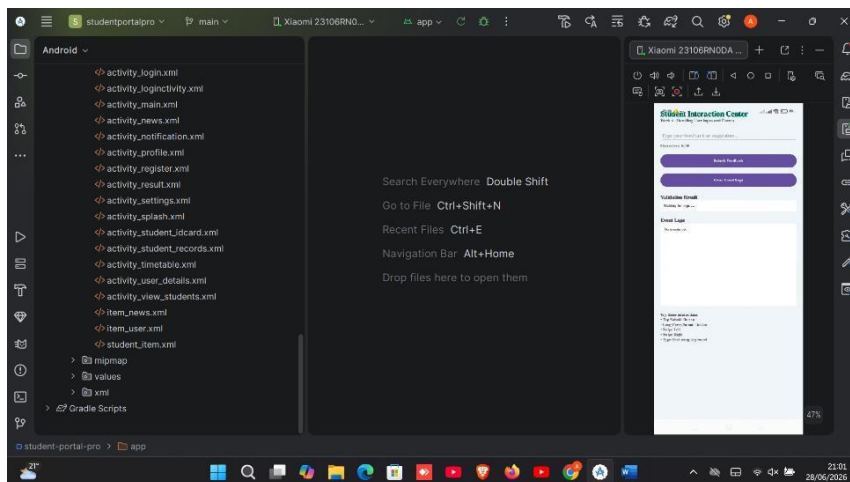
This week improved my understanding of mobile data storage and database integration. I learned how SQLite databases are used to store and manage application data locally. I implemented CRUD operations for student management and attendance tracking, which helped me understand how records are inserted, updated, retrieved and deleted. I also learned how Shared Preferences can be used to store small pieces of information such as login status. These activities strengthened my practical skills in developing mobile applications with persistent storage.

Week 8: Project Planning

Practical Activities Undertaken

- Developed an interaction module using Java classes and object-oriented programming.
- Implemented keyboard input validation and event handling.
- Added touch gesture support including tap, swipe, and long press events.
- Logged user interactions with timestamps.
- Improved the user interface with a character counter and feedback submission.

Interaction Feature on Dashboard



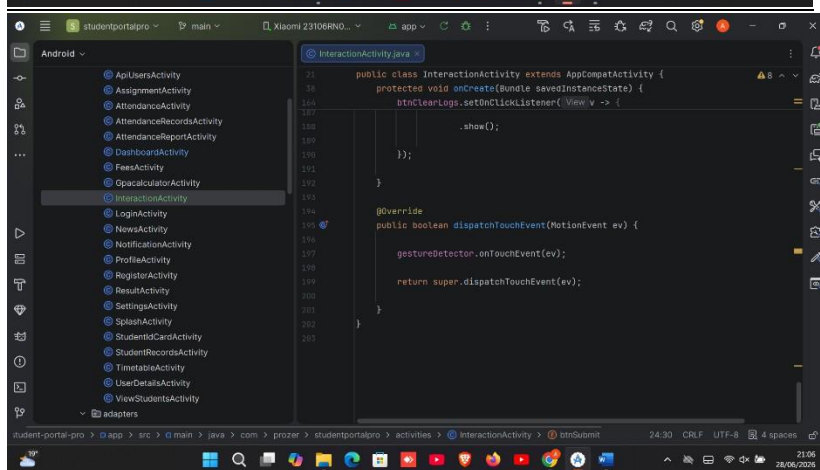
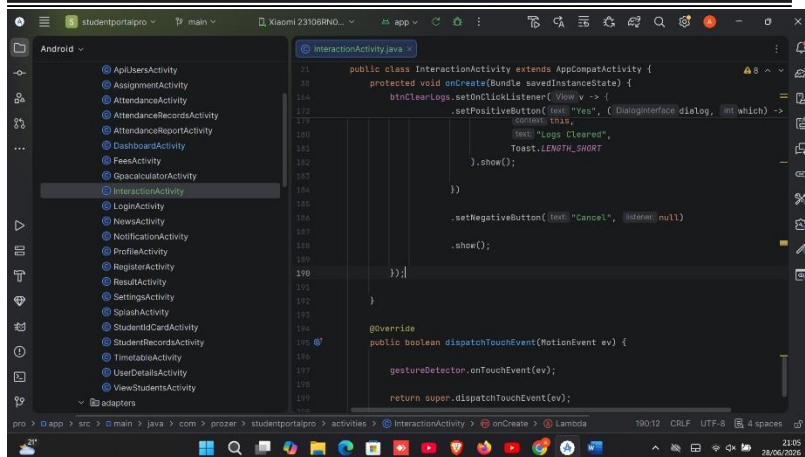
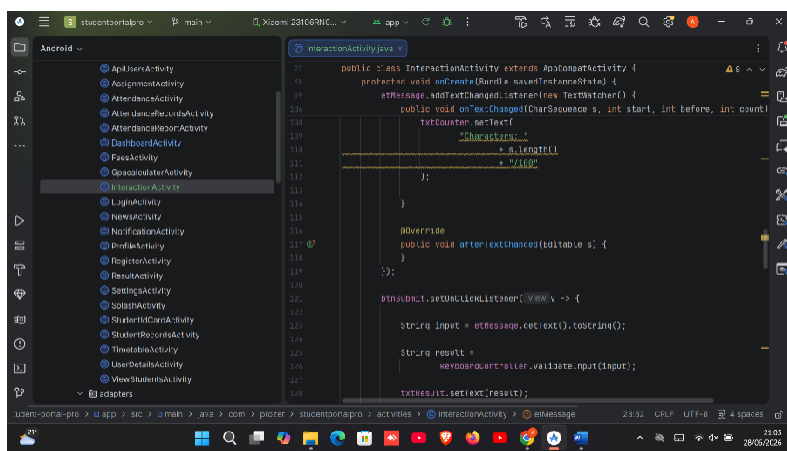
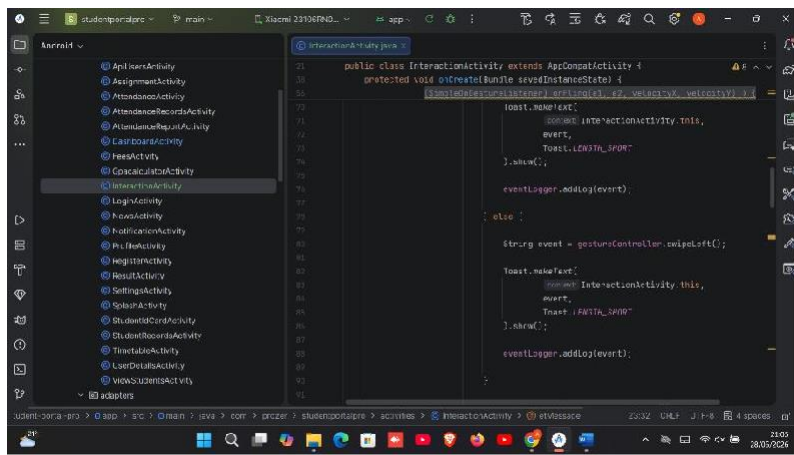


Figure 26 all codes on InteractionActivity.java

KeyboardController.java

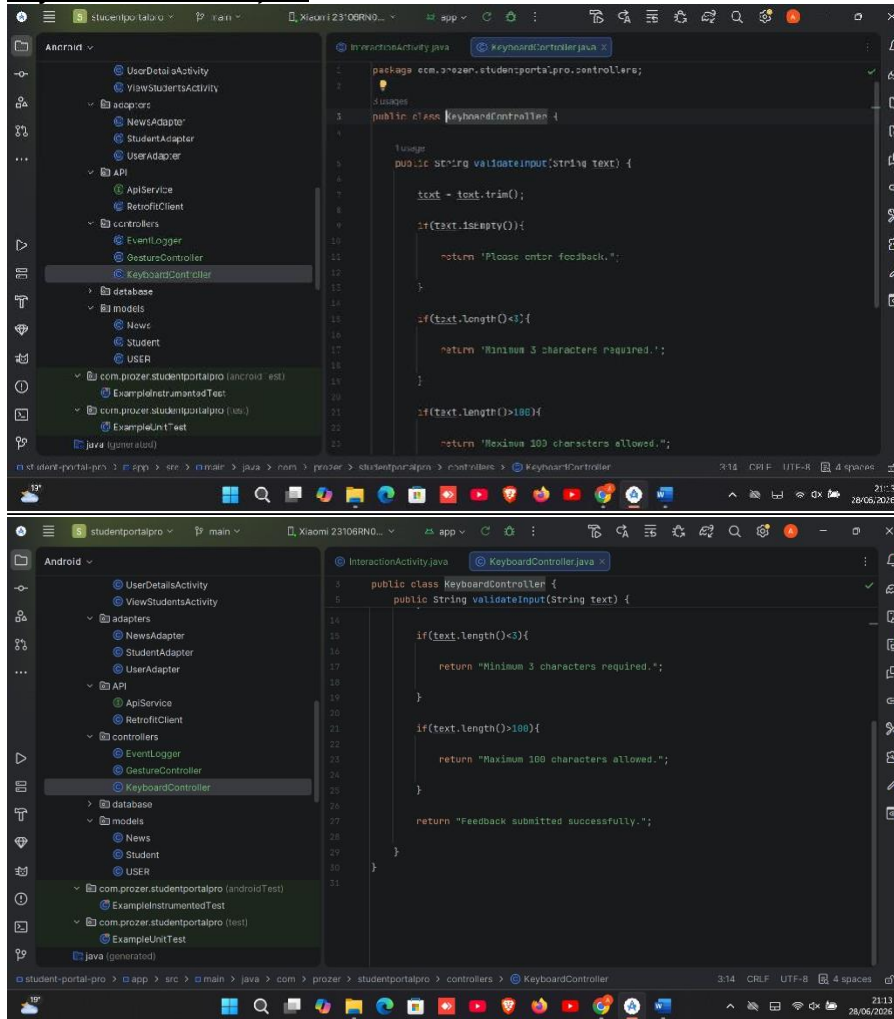


Figure 27 Keyboard Input Validation Class

GestureController.java

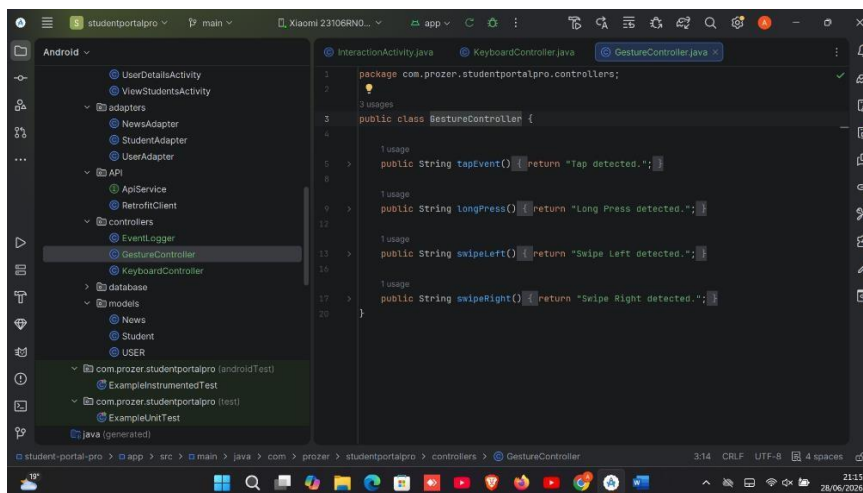


Figure 28 code on gesture control

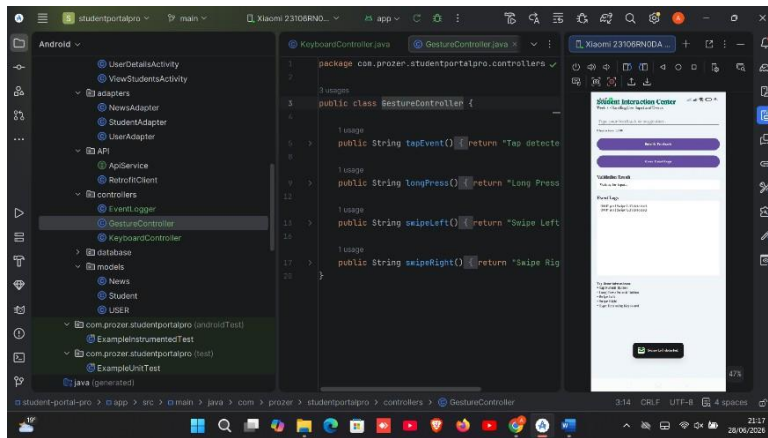


Figure 29 swipe gesture

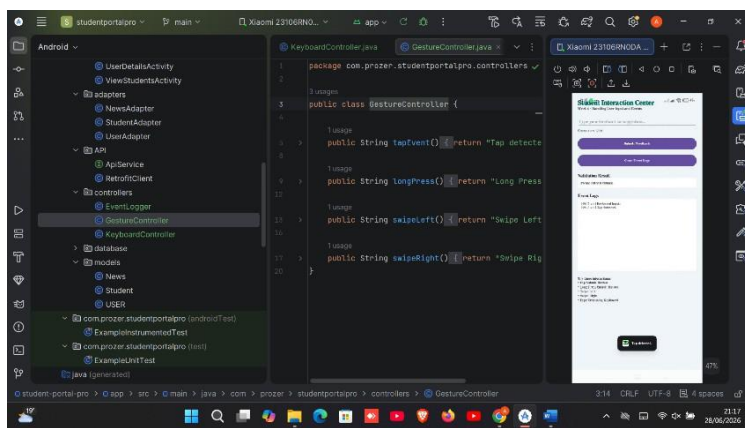


Figure 30 tap gesture

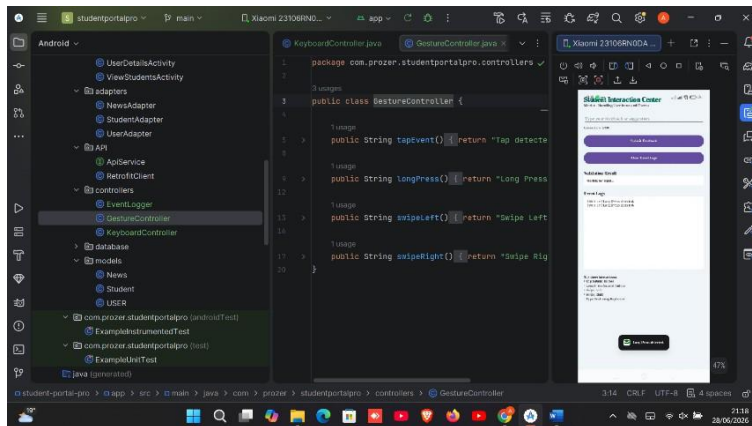


Figure 31 longpress gesture

EventLogger.java

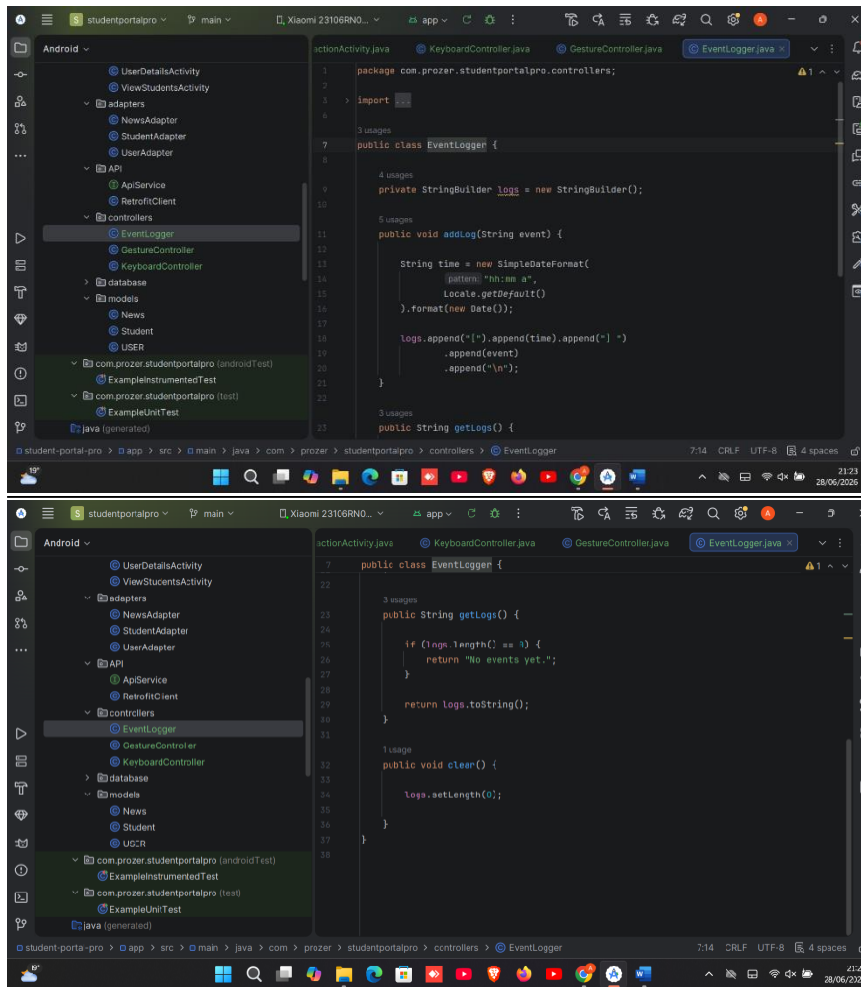
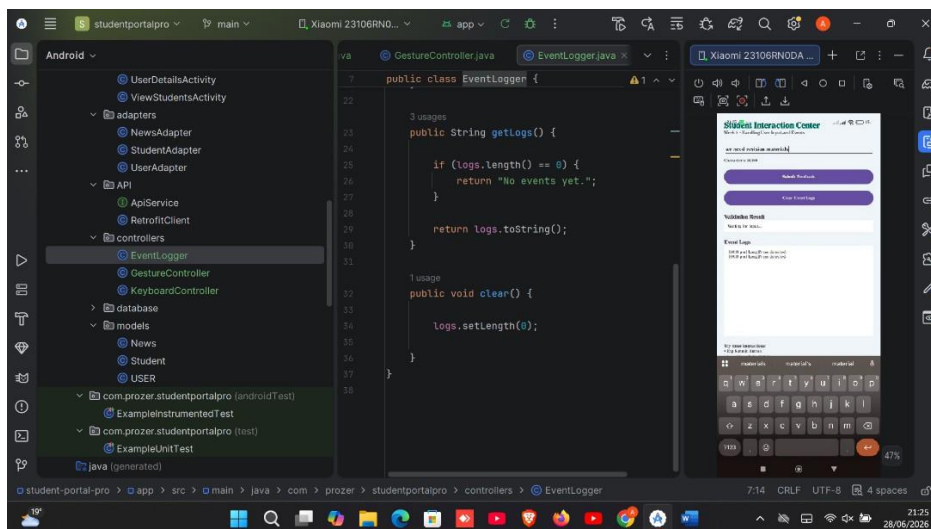


Figure 32 event logger.java code

Interaction Screen Running



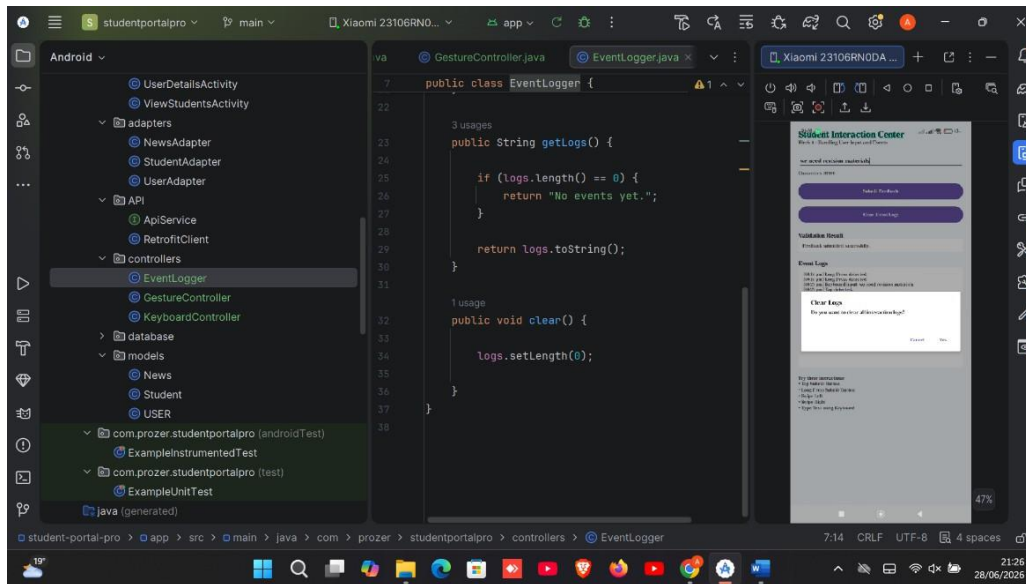


Figure 33 Interaction Screen Running

Tools Used

- Android Studio
- Java
- XML
- Android SDK
- RecyclerView
- Android Emulator

Personal Reflection

This week I implemented keyboard and touch event handling using object-oriented programming. I learned how to validate user input, detect gestures, and log interactions. These features improved my understanding of event-driven programming and made the Student Portal Pro application more interactive and user-friendly.

Week 9: Device Features and Sensors

Practical Activities

- Opened the **Campus Explorer** module from the dashboard.
- Launched the device camera and captured an image.
- Displayed the captured image successfully within the application.
- Retrieved the current GPS coordinates (latitude and longitude).
- Entered and saved the building information into the SQLite database.
- Viewed previously saved campus locations in the history screen.
- Tested the complete Campus Explorer workflow to ensure camera, GPS, data storage, and retrieval worked correctly without errors.

Dashboard home page

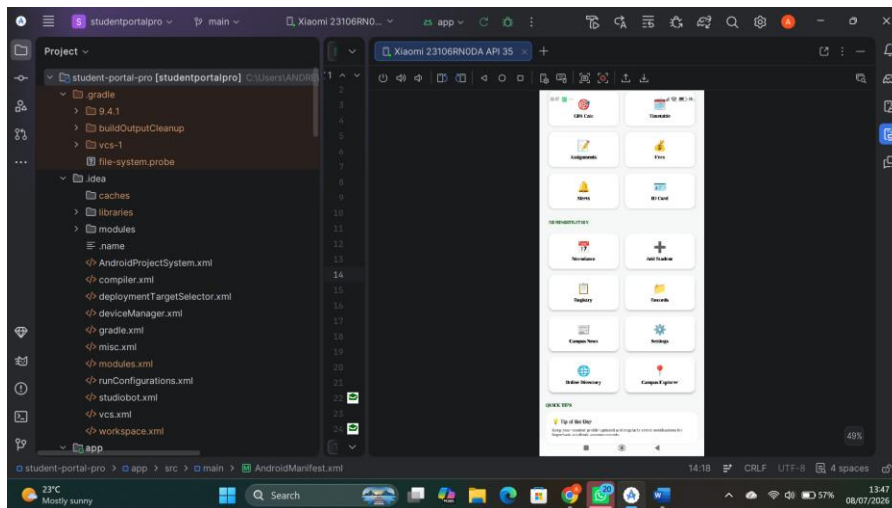


Figure34 Dashboard showing the newly integrated Campus Explorer feature.

Camera Open

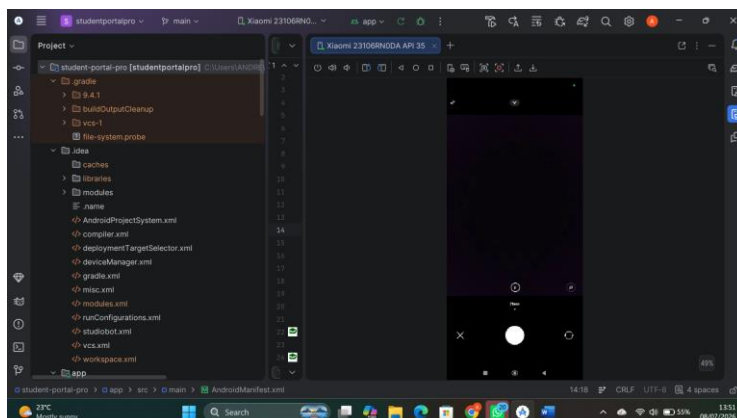


Figure 35 Camera successfully launched from the application.

Captured Image

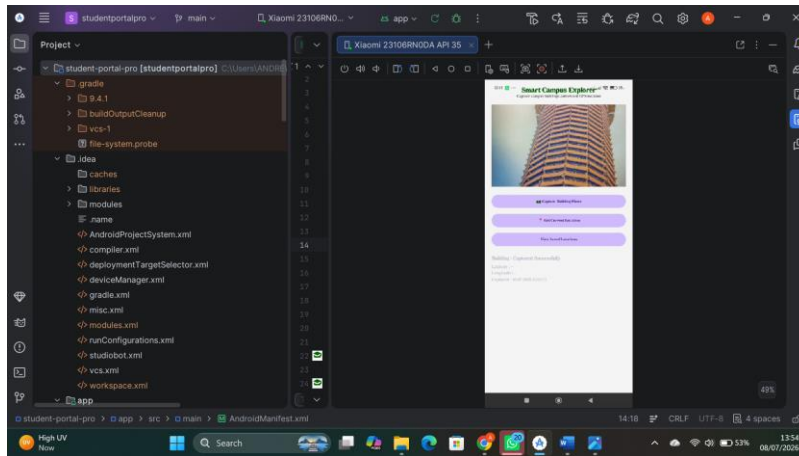


Figure 36 Image successfully captured and displayed.

Current GPS Location

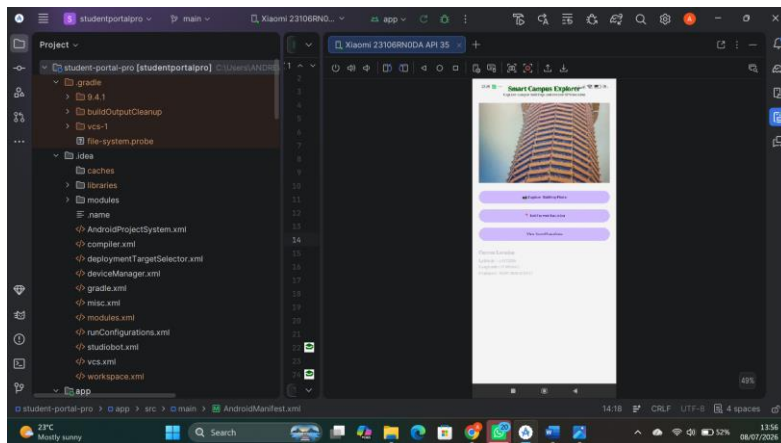


Figure 37GPS coordinates successfully retrieved.

Save Location

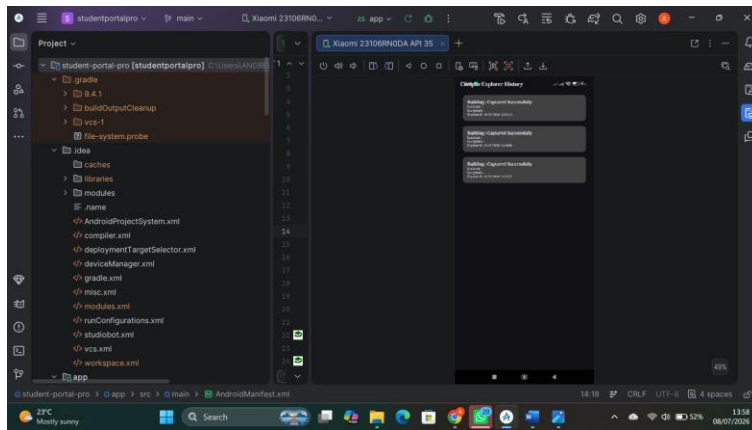


Figure 38 Campus location saved into SQLite database.

Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Camera API
- GPS (Location Services)
- Android Runtime Permissions
- Android Emulator / Physical Android Device
- Android SDK

PERSONAL REFLECTION

During Week 9, I integrated camera and GPS features into Student Portal Pro. I requested runtime permissions, captured images, retrieved GPS coordinates, and saved data in SQLite. Testing helped me resolve permission and storage issues, producing a stable and functional Campus Explorer module.

Practical Activities

- Integrated all application modules into one complete mobile application.
- Tested navigation between all screens to ensure proper functionality.
- Verified that dashboard statistics updated correctly after data changes.
- Improved the dashboard interface with a professional greeting, live date and time, and motivational quote.
- Performed functional testing and fixed minor errors to improve application performance and usability.

Final Dashboard

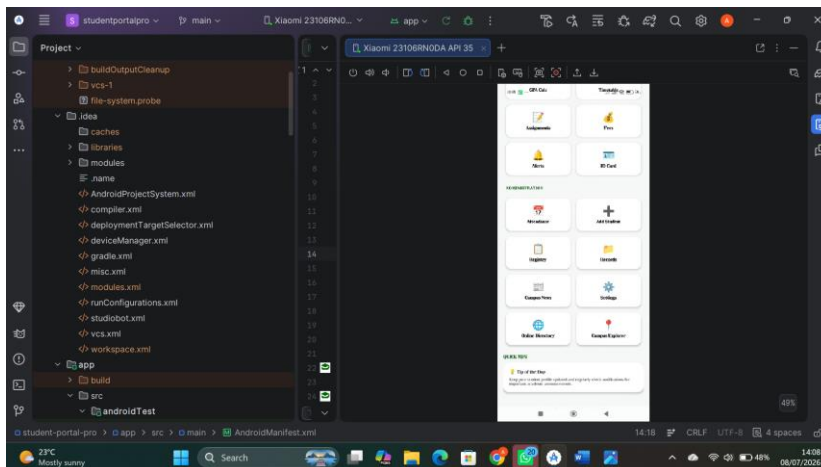
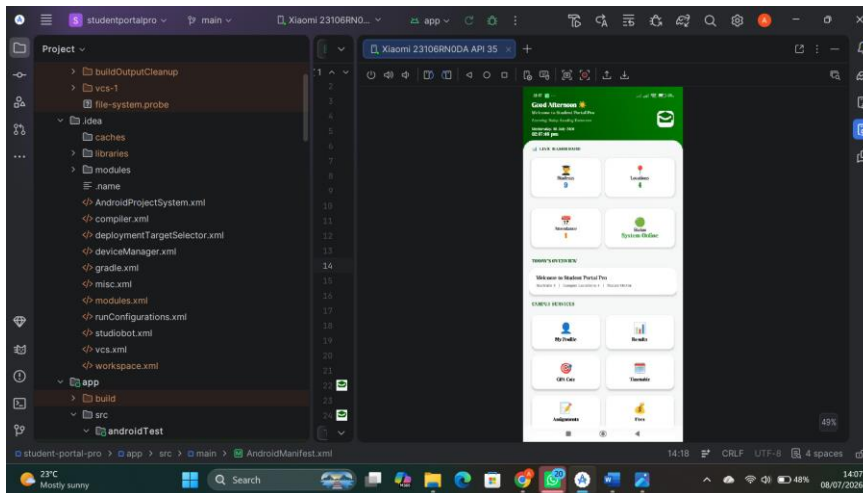


Figure 39 Final integrated dashboard showing application statistics and navigation.

Figure 40 Dashboard statistics updated automatically after integrating all application modules.

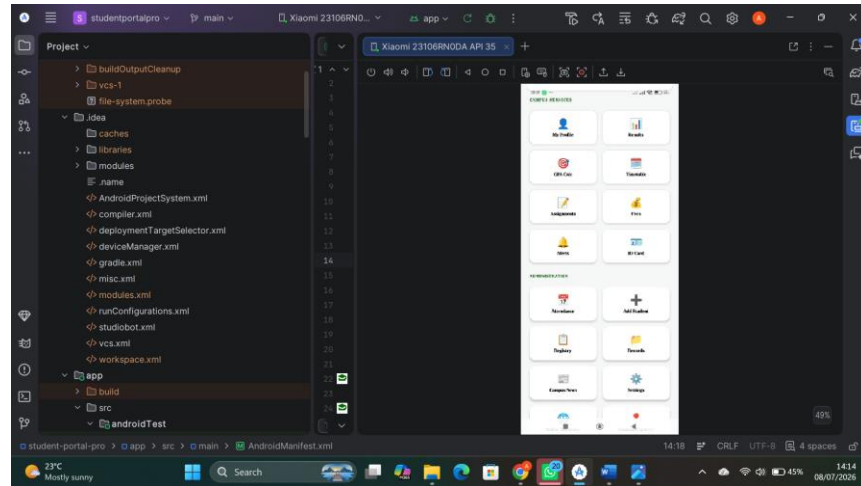


Figure 41 Integrated navigation connecting all application modules into one complete system.

- Android Studio
- Java
- XML
- SQLite Database
- RecyclerView
- Android SDK
- Android Emulator / Physical Android Device
- Logcat (Debugging Tool)
- Material Design Components

During Week 10, I integrated all application modules into one complete system, tested navigation and database functionality, improved the dashboard interface, fixed minor errors, and verified that the application operated smoothly with a professional user experience.

Week 11: Mobile Security

Practical Activities

- Updated the application version number and application information.
- Improved the splash screen with a professional loading interface.
- Updated the application launcher icon.
- Performed final testing of all application modules (Login, Dashboard, Student Management, Attendance, Camera, GPS and SQLite database).
- Verified smooth navigation between all application screens.
- Generated a release-ready APK for deployment.
- Prepared application screenshots and documentation for publishing.

Professional Splash Screen

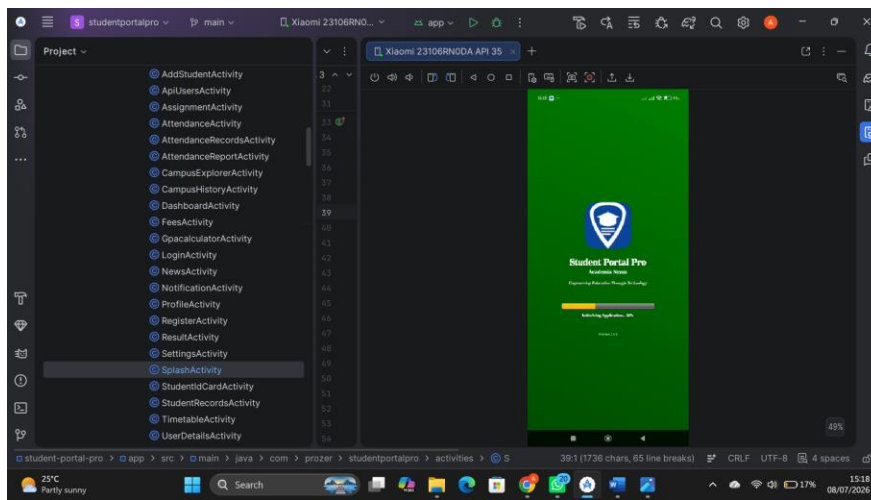


Figure 42 Professional Splash Screen

Final Dashboard

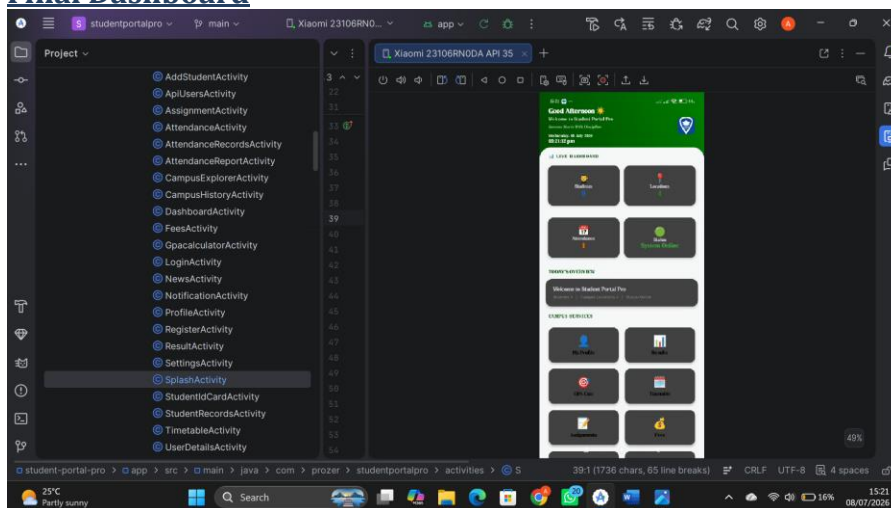


Figure 43 Final Dashboard

Final Testing

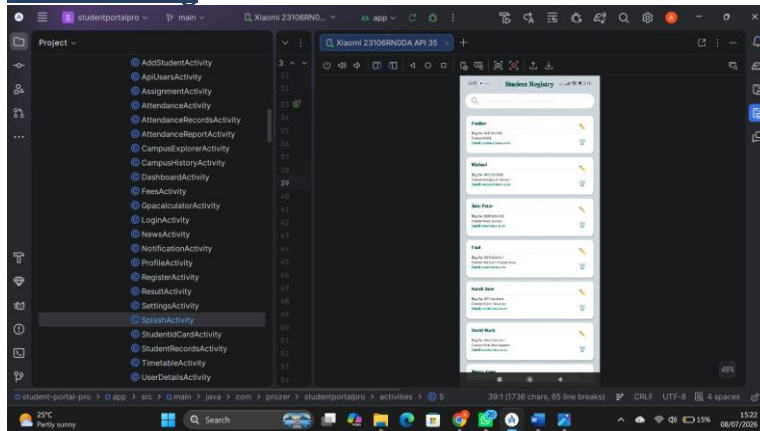


Figure 44 Proves that SQLite database is working and the application is functioning correctly.

Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Android SDK
- Gradle Build System
- Android Emulator / Physical Android Device
- Git (Project Backup)

Personal Reflection

I successfully prepared Student Portal Pro for deployment by improving its appearance, testing all features, generating a release APK, and updating application information. This week strengthened my understanding of software deployment, documentation, quality assurance, and preparing a professional Android application for end users.

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____